



中国科学院大学
University of Chinese Academy of Sciences

硕士学位论文

面向 ZYNQ SoC 平台嵌入式神经网络计算框架设计与实现

作者姓名: 李兴滢

指导教师: 尹震宇 研究员 中国科学院沈阳计算技术研究所

杨东升 研究员 中国科学院沈阳计算技术研究所

学位类别: 工程硕士

学科专业: 计算机技术

培养单位: 中国科学院沈阳计算技术研究所

2021 年 6 月

Design and Implementation of Neural Network Computing
Framework for ZYNQ SoC Embedded Platform

A thesis submitted to

University of Chinese Academy of Sciences

in partial fulfillment of the requirement

for the degree of

Master of Engineering

in computer technology

By

Li Xingying

Supervisor: Professor Yin Zhenyu

Professor Yang Dongsheng

Shenyang Institute of Computing Technology

Chinese Academy of Sciences

June 2021

摘 要

随着深度学习领域的发展以及智能边缘设备的迅速普及，在移动嵌入式设备上实现深度学习算法的需求越来越大，神经网络的研究成果不断落地于实际的生产、生活和服务，逐渐推广到嵌入式平台。将神经网络技术引入到具有资源受限的嵌入式平台，已成为未来深度学习技术进步和发展的主要趋势。然而，由于追求更高的精度，现有神经网络计算框架在功能越来越丰富的同时伴随着更复杂的运算，大量的内存需求和计算能力需求使得神经网络计算框架在资源受限、计算能力有限的嵌入式平台上部署成为难点。从而导致神经网络技术很难适应工业应用需求，在嵌入式领域的发展受到一定制约。因此，搭建性能优异且功耗较低的嵌入式神经网络计算框架是推进神经网络技术在嵌入式平台应用的关键。

本文以 ZYNQ SoC 作为开发平台，首先实现了 ZYNQ SoC 嵌入式平台的 Linux 操作系统移植，并设计实现了图像采集输入模块，以构建完整的神经网络应用系统。其次，针对现有的神经网络计算框架计算复杂、占用存储资源多的问题，本文对开源神经网络框架 DarkNet 进行扩展和改进，引入了深度可分离卷积算法以及轻量神经网络模型 MobileNetV2。最后，针对框架中 GEMM 算法计算量大、耗费资源多的问题，使用了 ARM NEON 汇编指令以及 OpenBLAS 对该算法进行了优化，得到了面向 ZYNQ SoC 的嵌入式神经网络计算框架。通过实验对比，证实了改进后的框架在计算效率方面有较大提升，并且在一定程度上降低了资源消耗。本文所做的贡献和创新点如下：

1) 为了使设计的框架能够应用到实际场景中，本文设计并实现了图像采集输入单元。针对 ZYNQ SoC 裸机环境开发难度大的问题，采用面向操作系统开发的方法，实现了 Linux 操作系统的 U-BOOT、Kernel 以及 ramdisk 配置和移植。建立了神经网络计算框架对数字图像进行处理和分类的完整系统。通过实验可以得出，该模块采集的图片可以提供给神经网络进行处理，在本文改进的框架中进行分类识别，并且保持了原有的识别精度。

2) 为了解决嵌入式平台资源有限、算力低的问题, 本文对轻量级开源神经网络框架 DarkNet 进行改进, 在其基础上引入了深度可分离卷积算法, 并结合该算法实现了高效的轻量神经网络模型 MobileNetV2。极大降低了卷积计算的复杂度和参数量, 在保持准确度的同时尽可能减少了存储资源和算力需求。通过实验进行对比, 可以看出使用可分离卷积的 MobileNetV2 网络模型具有很好的图像分类能力, 并且占用资源较少、计算效率高。

3) 为了解决卷积计算量大, 占用资源多的问题, 结合 ZYNQ SoC 具备 ARM Cortex-a9 双核的优势, 本文针对卷积计算中的核心算法 GEMM 矩阵乘法运算进行 ARM NEON 汇编指令优化, 实现数据读取、矩阵相乘设计等运算; 最后实现了利用 OpenBLAS 对 GEMM 的优化。通过测试, 可以看出, 这两种方式都能有效提高卷积计算的计算效率, 较好的提升了 ZYNQ SoC 平台神经网络计算的实时性。

通过在 ZYNQ SoC 开发板上使用本文改进后的神经网络计算框架进行图像分类实验, 计算速率得到了极大地提升, 在保证了一定识别精度的同时有效降低了存储开销和算力需求, 基本满足于实际应用需求。

关键词: ZYNQ SoC, 神经网络框架, 卷积计算优化, ARM NEON, OpenBLAS

Abstract

With the development of deep learning field and the rapid popularity of intelligent edge devices, there is an increasing demand for implementing deep learning algorithms on mobile embedded devices. The research results of neural networks continue to land in actual production, life and services, and are gradually promoted. Introducing neural network technology into embedded platform with limited resources has become the main trend of deep learning technology progress and development in the future. However, due to the pursuit of higher precision, the existing neural network computing framework is accompanied by more and more complex operations with richer functions. The large memory requirements and computing power requirements make the deployment of neural network computing framework on the embedded platform with limited resources and computing power become difficult. As a result, the neural network technology is difficult to adapt to the needs of industrial applications, and its development in the embedded field is restricted to a certain extent. Therefore, building an embedded neural network computing framework with excellent performance and low power consumption is the key to promote the application of neural network technology in embedded platform.

In this paper, ZYNQ SoC is used as the development platform. To construct a complete neural network application system, the Linux operating system is transplanted to the ZYNQ SoC embedded platform first, then the image acquisition input module is designed, so as to achieve the purpose of flexible development and experimental verification in real application scenarios. Secondly, in view of the existing neural network computing framework is complex and takes up a lot of storage resources, this paper expands and improves the open source neural network framework Darknet, and introduces the depthwise separable convolution algorithm and the lightweight neural network model MobileNetV2. Finally, aiming at the problem of large computation and resource consumption of GEMM algorithm in the framework, ARM NEON assembly

instruction and OpenBLAS were used to optimize the algorithm, and an embedded neural network computing framework oriented to ZYNQ SoC was obtained. The contributions and innovations of this paper are as follows:

1) In order to apply the framework to actual scene, this paper designs and implements the image acquisition input unit. To solve the difficulty of ZYNQ SoC bare-metal environment development problem, this paper adopts the method of operating system oriented development. The U-BOOT, kernel compilation and ramdisk root file system transplantation are realized for the Linux operating system. A complete system for processing and categorizing digital images with a neural network computing framework is established. Through experiments, it can be concluded that the pictures collected by this module can be provided to the neural network for processing, and the classification and recognition are carried out in the improved framework of this paper, and maintaining the original recognition accuracy.

2) To tackle the problem of limited resources and low computing power of the embedded platform, this paper improves the lightweight open source neural network framework Darknet, introduces the deep separable convolution algorithm on the basis of Darknet, and realizes the efficient lightweight neural network model MobileNetV2 combined with this algorithm. It greatly reduces the complexity and number of parameters of convolution computation, and reduces storage resources and computing force requirements as much as possible while maintaining accuracy. Through the comparison of image classification experiments, we can see that the MobileNetV2 network model has a good ability of image classification and occupies less resources with high computational efficiency.

3) In order to solve the problem of large amount of convolution calculation and high resource consumption, combined with the advantages of the ARM Cortex-a9 dual-core of the ZYNQ SoC, this paper optimized the core algorithm of convolution computation, GEMM matrix multiplication operation, carried out assembly instruction optimization of ARM NEON, realized data reading, matrix multiplication design and

other basic operations. Finally, the optimization of GEMM using OpenBLAS function library is realized. Through the test, it can be seen that both of these two methods can effectively improve the computational efficiency of convolutional computation, and can better improve the real-time performance of embedded platform neural network computation.

By using the improved neural network computing framework in this paper on the ZYNQ SoC development board to conduct image classification experiments, the computing rate has been greatly improved, which ensures a certain recognition accuracy and effectively reduces the storage overhead and computing force requirements, basically meeting the practical application requirements.

Key words: ZYNQ SoC, Neural Network Framework, Convolutional Computing Optimization, ARM NEON, OpenBLAS

目 录

第 1 章 绪论.....	1
1.1 选题的背景和意义.....	1
1.2 国内外相关领域的研究现状.....	2
1.2.1 神经网络计算框架研究现状.....	2
1.2.2 神经网络模型研究现状.....	3
1.3 研究内容和组织结构.....	4
1.3.1 研究内容.....	4
1.3.2 组织结构.....	5
1.4 本章小结.....	6
第 2 章 神经网络原理及相关技术研究	7
2.1 绪论.....	7
2.2 神经网络.....	7
2.2.1 基本概念和思想.....	7
2.2.2 神经网络结构.....	8
2.3 卷积神经网络.....	11
2.3.1 卷积神经网络基本结构.....	11
2.3.2 卷积神经网络的特性.....	15
2.4 卷积计算加速技术研究.....	16
2.4.1 基于模型的优化方法.....	16
2.4.2 基于计算框架的优化方法.....	18
2.5 本章小结.....	19
第 3 章 基于 ZYNQ Linux 的图像采集模块构建.....	21
3.1 绪论.....	21
3.2 ZYNQ SoC 硬件平台介绍.....	21
3.3 硬件环境搭建.....	22
3.4 软件环境开发.....	23

3.4.1 编译环境搭建.....	24
3.4.2 U-Boot 配置.....	24
3.4.3 Linux kernel.....	25
3.4.4 设备树文件.....	26
3.4.5 文件系统.....	27
3.5 系统移植.....	27
3.6 OV5640 图像采集模块.....	28
3.6.1 VDMA	29
3.6.2 FPGA 工程配置	30
3.7 本章小结.....	32
第 4 章 框架设计与实现	33
4.1 绪论.....	33
4.2 基于 DarkNet 框架的卷积计算优化.....	33
4.2.1 DarkNet 框架简介	33
4.2.2 DarkNet 中的卷积计算研究.....	34
4.2.3 深度可分离卷积算法实现.....	35
4.3 构建轻量神经网络模型.....	38
4.3.1 MobileNetV2 网络结构	39
4.3.2 反向残差结构.....	40
4.3.3 线性瓶颈结构.....	42
4.3.4 MobileNetV2 的实现	43
4.4 GEMM 计算的 NEON 优化.....	45
4.4.1 ARM Cortex-a9 微处理器	46
4.4.2 基于 NEON 汇编指令的 GEMM 实现.....	46
4.5 GEMM 计算的 OpenBLAS 加速实现	49
4.6 本章小结.....	52
第 5 章 测试与分析	53
5.1 实验设计.....	53
5.1.1 开发环境部署.....	53
5.1.2 图像分类测试集.....	54

5.1.3 评价指标.....	55
5.2 板上测试及分析.....	55
5.2.1 功能性测试.....	56
5.2.2 性能测试.....	57
5.2.3 执行时间测试.....	57
5.3 本章小结.....	58
第 6 章 总结与展望	59
6.1 工作总结.....	59
6.2 展望.....	60
参考文献.....	61
致 谢.....	65
作者简历及攻读学位期间发表的学术论文与研究成果	67

图表目录

图 2.1 神经元对应关系.....	8
图 2.2 神经元模型.....	9
图 2.3 Sigmoid 及 ReLu 函数图像	10
图 2.4 多层感知器.....	11
图 2.5 LeNet 的网络结构.....	12
图 2.6 卷积计算.....	13
图 2.7 最大池化示例.....	14
图 3.1 硬件配置.....	23
图 3.2 软件开发流程.....	23
图 3.3 BOOT.BIN 创建	28
图 3.4 系统启动界面.....	28
图 3.5 图像采集模块架构.....	29
图 3.6 图像采集底层设计.....	30
图 4.1 im2col 实现原理	34
图 4.2 标准卷积.....	36
图 4.3 深度可分离卷积.....	37
图 4.4 MobileNetV2 结构	39
图 4.5 线性瓶颈块.....	40
图 4.6 网络层计算方法.....	41
图 4.7 网络结构对比.....	42
图 4.8 ReLU6 激活函数	44
图 4.9 训练参数配置.....	45
图 4.10 性能分析.....	45
图 4.11 SISD 与 SIMD 对比	47
图 4.12 NEON 寄存器	48
图 4.13 NEON 加速原理	49
表 4.1 网络层计算表达式.....	41
表 4.2 MobileNetV2 网络结构	44

表 4.3 NEON 数据类型	47
表 5.1 正确率测试结果.....	56
表 5.2 性能对比.....	57
表 5.3 速度评估.....	58

第1章 绪论

1.1 选题的背景和意义

当前，由于智能设备的兴起，提供了越来越多可用的数据，加上硬件计算能力的提高，使得神经网络取得了重大进展。数据和计算能力都使得神经网络可以解决的任务变得越来越广泛。机器学习，尤其是其子领域深度学习取得了令人瞩目的进步。在这些领域内开发的技术现在能够学习和分析大量不同格式的真实示例，计算机通过对这些实例的分析，可以获得对数据的洞察，从而为正在分析的问题提供有用信息。如进行价值预测，语音和图像处理与识别，自然语言理解，情感分析，财务战略，基因映射，欺诈检测，翻译等。当从事这些激动人心的项目时，我们通常希望将乏味的工作“外包”，例如编写模型算法到深度学习框架。

机器学习算法的数量广泛且不断增长，使得通过框架和库的实现也越来越广泛。深度学习框架正在推动人工智能革命，没有它们，数据科学家几乎不可能在他们的深度学习算法中提供先进的水平，从而使计算和处理能力的进步成为可能。神经网络计算框架以组件的形式对神经网络的底层算法进行构建，使用者只需根据网络结构就可以快速构建神经网络模型，减少了重复造轮子的工作，极大简化了神经网络的开发流程（谢启荣, 2019）。简而言之，深度学习框架使构建相当复杂的深度学习算法变得更加容易，简化困难的任务并最大程度地减少艰巨的任务。

随着物联网和智能边缘设备的迅速普及，在移动嵌入式设备上实现包括深度学习在内的机器学习算法的需求越来越大（Meng et al., 2017），神经网络领域的研究成果不断落地于实际的生产、生活和服务，神经网络技术逐渐推广到嵌入式平台，带来了重大改进，以便在边缘实现智能和自主。如工厂车间的自动化机械和工业机器人，到家中的自动吸尘器，再到田间的农用拖拉机，并可能给无数行业带来同样的改进。与传统的机器学习相比，神经网络算法可以提供更高的准确性、更强的通用性和更好的数据利用率。然而，为了取得更高的

精度，现有神经网络计算框架在功能越来越丰富的同时伴随着更复杂的运算和结构，也带来了更多的能耗。尽管这些网络功能非常强大，但由于大量的内存和计算能力需求使得它们在资源受限、计算能力有限的嵌入式平台上部署成为难点（李世明, 2018），严重阻碍了嵌入式神经网络技术在实际生活中的应用和发展。目前，一些深度学习应用程序被部署在有 GPU 的嵌入式系统上，如智能手机和平板电脑等，这些应用程序大多数仍然依赖于通过互联网将数据上传到服务器进行计算，然后下载结果。但是，这种基于云的客户机-服务器模型存在很高的延迟、网络开销以及隐私问题（Qian et al., 2017），并且嵌入式系统中通常不存在 GPU，这使得神经网络难以适用于实际业务应用场景，服务更多行业，实现其更大的价值。由此可见，平衡算力与开销，在嵌入式移动系统上搭建性能优异的嵌入式神经网络计算框架，部署离线轻量神经网络模型，是推进神经网络在嵌入式平台上部署和应用的关键。针对嵌入式平台中神经网络计算的低成本、低功耗、低延迟的“三低”需求，本文面向 ZYNQ SoC 平台，在开源神经网络框架 DarkNet 的基础上，引入了深度可分离卷积、优化了矩阵运算，并部署了轻量神经网络模型 MobileNetV2，以提高嵌入式平台神经网络可用性和计算效率。

1.2 国内外相关领域的研究现状

1.2.1 神经网络计算框架研究现状

自 2012 年深度学习取得了显著的成果以来，机器学习框架便逐渐成为学术界研究的热点，许多学术团体和行业团体启动了一些计划来构建强大的深度学习库和框架，从早期的学术成果 Caffe 和 Theano 到庞大的行业支撑的 PyTorch 和 TensorFlow，得到了广泛地使用，这些框架有助于轻松创建和测试不同的深度架构（Bahrampour et al., 2016）。由 Google Brain 团队创建的 TensorFlow（Abadi et al., 2016）是当今使用最广泛的开源深度学习框架之一，为机器学习和深度学习提供了强大的支持，其灵活的体系结构允许在本地计算机，云中的集群，iOS 和 Android 设备，CPU 或 GPU 之间轻松部署计算。PyTorch（Paszke et al., 2017）可以执行高复杂度的张量计算，除了使用 Python 进行外，还具有硬

件加速组件、高度交互的开发模型、按需设计工作，并且已经包含许多实用的组件。对于快速开发而言，PyTorch 通常是更好的选择，但是在大型的项目以及复杂的计算中 TensorFlow 表现更突出。Caffe 以其类似激光的速度而闻名，受 C、C++、Python、MATLAB 和 Command Line 等接口支持，它在 CNN 建模中具有普适性及高效性。但是，Caffe 在给定的体系结构下，总体支持度低，对于复杂层类型的创建需要使用低级语言来完成。

以上这些框架依赖项多，缺少对嵌入式设备的支持。随着神经网络技术不断向嵌入式平台延伸，近几年学术界提出了许多面向嵌入式平台的轻量神经网络计算框架的相关研究。由谷歌公司设计的 TensorFlow Lite（李双峰, 2020）是 TensorFlow 工具的轻量级解决方案，具有简化的操作集，适用于移动和嵌入式设备。由于缺乏清晰的文档，并且内置运算符库非常有限、自定义操作符难度高，导致 TensorFlow Lite 导入模型容易出错，一些模型（如 ResNet（He et al., 2016））的操作不被支持，应用困难。Facebook 推出的 Caffe2 考虑了表达，速度和模块性，它继承了 Caffe（Jia et al., 2014a）的许多设计，同时解决了多年来使用和部署 Caffe 时发现的瓶颈。Caffe2 的核心库提供了可移植性，而其 Python 和 C++ API 使其可以轻松地进行原型设计，训练和部署。但是该框架并未针对移动端进行计算优化，在移动端的计算速度很慢（李全, 2019）。小米公司推出的移动 AI 计算引擎 MACE（赵群 等, 2020），其核心用 C++ 实现，依赖于底层硬件。与 TensorFlow Lite 以及 Caffe 2 相比，支持异构计算是该框架的一个优势，缺点是速度不够理想。

1.2.2 神经网络模型研究现状

神经网络算法是功能强大且流行的算法。他们的许多成功都在于其体系结构的精心设计。（Lecun and Bottou, 1998）提出了一种用于字符识别的神经网络架构 LeNet-5。与当今的深度学习神经网络相比，它具有非常简单的体系结构构建和更少的层数（戴雷燕 等, 2019）。该网络是许多模型的起源，有效推动了深度学习领域的发展。2012 年，Alex Krizhevsky 发布的仅使用 8 层 CNN 的 AlexNet（Krizhevsky et al., 2012）在艰难的 ImageNet 大规模视觉识别挑战赛中大获全胜。AlexNet 将 LeNet 的思想扩展为一个更大的神经网络，能够进行更复杂结构

的学习。AlexNet 的主要功能包括使用 ReLU 代替 tanh 函数、针对多个 GPU 的优化以及重叠池、使用数据扩充和删除来解决过拟合问题。该网络首次表明，通过学习获得的功能可以超越手动设计的功能，从而打破了计算机视觉的先前范式，是一项革命性的进步。牛津大学的 VGG (Simonyan and Zisserman, 2014) 网络最大的优点在于多重的 3×3 卷积序列可以洞察更大的感受野的效果。VGG 证明了更深层次的网络可以带来更好的结果，但是该网络通常很大，包含约 1.6 亿个参数，增加了计算成本。

为了使神经网络技术成果转化为工业界的产品和服务，在资源受限的嵌入式设备上应用，近年来，轻量化卷积神经网络模型成为了学术界研究的热点。SqueezeNet (Iandola et al., 2016) 在保持分类性能的同时，减少了参数数量和计算量。Xception (Chollet, 2017) 中首次引入了深度可分离卷积，它们是大多数轻量级架构的主要组成部分，为后续轻量化模型提供了思路。MobileNet (Howard et al., 2017) 是利用深度可分离卷积构造而成的轻量级深度神经网络，还提供了宽度因子和分辨率因子两个参数，进一步减少其操作数量。ShuffleNet (Zhang et al., 2017) 使用了组卷积大大降低了计算成本，并且使用通道混洗来减少信息损失。MobileNetv2 (Sandler et al., 2018) 利用了反向残差结构，其中中间扩展层使用深度卷积。ShuffleNetV2 (Ma et al., 2018) 构建在 ShuffleNet 的基础上，使用了通道分割和通道混洗，实现了一个特性重用模式 (Kopuklu et al., 2019, 毕鹏程 等, 2019)。这些架构充分利用了组卷积或深度可分离卷积的思想，有效提升了 CNN 在卷积层的计算速率。

1.3 研究内容和组织结构

1.3.1 研究内容

本文以 DarkNet 开源框架为基础，对其中的卷积运算以及矩阵乘法运算进行分析，研究神经网络计算中提高卷积计算速度、降低卷积计算参数的优化方法，最终实现神经网络计算框架直接应用在基于 Linux 操作系统的 ZYNQ SoC 嵌入式平台上的目标。具体的研究内容如下：

- 1) 实现基于 ZYNQ 平台的图像采集输入模块。

本文为了建立神经网络计算框架对数字图像进行处理和分类的完整系统，基于 ZYNQ SoC 嵌入式移动平台设计并实现了图像采集输入模块，并针对嵌入式平台裸机环境开发难度大的问题，采用面向操作系统开发的方式，将 Linux 操作系统移植到 ZYNQ SoC 嵌入式移动平台。旨在充分利用操作系统的灵活性和便捷性，保证本文所实现的神经网络计算框架可在实际应用场景下使用。

2) 实现深度可分离卷积，构建了轻量神经网络模型 MobileNetV2。

本文对当前神经网络计算框架和计算模型的轻量化方法进行了研究与分析，对开源神经网络计算框架 DarkNet 进行改进和优化，实现了当前多数轻量神经网络模型都使用的深度可分离卷积算法，并引入了带有深度可分离卷积的线性瓶颈块所构成的高效轻量神经网络计算模型 MobileNetV2。从而达到降低神经网络计算复杂度以及参数量，提高神经网络技术在嵌入式平台应用的高效性。

3) 实现对 GEMM 矩阵乘法的 NEON 汇编优化以及 OpenBLAS 函数库优化。

本文通过使用 perf 软件对 DarkNet 框架中各函数计算速度以及占用资源情况的分析，找到影响神经网络运算的关键算法 GEMM 矩阵乘法运算，该运算占据了神经网络运算的大部分时间和资源。故本文结合了 ZYNQ SoC 具备的 ARM Cortex-a9 双核优势，实现了 ARM NEON 指令以及 OpenBLAS 对 GEMM 算法的优化。

4) 对改进后的神经网络计算框架进行测试与分析

本文使用 ZYNQ SoC 作为嵌入式开发平台，在真实的场景下对改进和优化后的神经网络计算框架进行采集图像的分类测试。首先，使用实现的模型对采集单元采集的图片进行图像分类精确度测试；其次，将使用了可分离卷积思想的 MobileNetV2 与其它未使用深度可分离卷积的模型进行单幅图像分类进行对比，评估不同框架对单幅图片分类的精确度以及耗时等性能；最后，将框架中利用未优化过 GEMM 算法的神经网络模型与优化 GEMM 后的神经网络模型对单幅图像的识别速率进行了对比和分析。

1.3.2 组织结构

本文共六个章节，每个章节及其主要内容介绍如下：

第一章，本章节首先对选题的背景及该研究的意义进行了介绍，然后对本

课题所涉及的相关领域技术以及国内外研究现状进行了阐述。

第二章，本章节对本课题相关的神经网络概念和技术进行了详细介绍，为神经网络计算框架的设计实现以及卷积计算的优化做了铺垫。

第三章，本章节对 ZYNQ SoC 嵌入式平台的架构进行了研究与分析，基于 ZYNQ SoC 设计并实现了 Linux 嵌入式操作系统以及图像采集输入单元。

第四章，本章节研究分析了 DarkNet 中的卷积计算方式，以深度可分离卷积替代传统卷积进行了卷积计算优化，并实现了高效且轻量的神经网络模型 MobileNetV2。最后研究分析了 ARM NEON 指令的优势以及 OpenBLAS 的使用，实现了 GEMM 矩阵乘法计算的两种优化。

第五章，本章节对改进后的框架进行了可用性、准确性、实时性等测试，并对测试结果进行了分析与总结。

第六章，本章节对全文工作进行了总结，并对未来的研究和改进的方向进行了充分的展望。

1.4 本章小结

本章主要介绍了嵌入式神经网络技术发展的背景，说明了嵌入式神经网络计算框架的设计和实现具有重要的研究意义。其次，总结了国内外对于神经网络计算框架及神经网络模型的研究现状，明确了本文的研究方向以及研究目标。最后介绍了本文的研究内容和组织结构。

第2章 神经网络原理及相关技术研究

2.1 绪论

为了实现嵌入式平台的神经网络框架并提升其计算性能，本章节首先对神经网络的基本概念和思想进行了详细介绍，接着介绍了深度学习中一种重要的神经网络——卷积神经网络，最后对该网络的相关加速方法进行了研究和分析。通过对神经网络原理及相关技术的研究，为嵌入式神经网络计算框架的实现提供了理论基础。

2.2 神经网络

2.2.1 基本概念和思想

神经网络是一系列算法，基本上是对大脑进行计算机模型化的一种尝试。每个神经网络都充当一个神经元，旨在识别模式，分组和分析在现实世界中收集的数据，例如图像，声音，文本和时间序列。神经网络，可能是人工智能（AI）领域最先进和最具挑战性的基础，它正在对许多应用程序产生重大影响，使产品能够像人类一样智能地运行。二十多年前，神经网络被广泛视为下一代计算技术，最终将使计算机能够自己思考。现在，由于硬件的改进和软件模型的改进，围绕该技术的思想（大致基于哺乳动物大脑如何学习的生物学知识）开始渗入主流计算。尤其是在图像识别和分类，语音识别，文本分析和虚拟助手领域。当然，计算机仍然不能自己思考，但是神经网络的最新创新使计算机可以在广阔的数据领域中进行筛选，并在没有人工帮助的情况下得出基本结论。

神经网络与传统计算的不同之处在于：在传统计算中，计算机被赋予一个特定的算法或程序来执行。而通过神经网络，解决特定问题的工作很大程度上取决于计算机自身。为了解决一个问题，比如在一个背景下找到一个特定的物体，神经网络使用了一种与哺乳动物大脑皮层运作方式相似的，但极为简化的方法。大脑利用数十亿相互连接的神经元来处理感觉和其他信息，随着时间的推移，神经元之间的联系发生了变化，人对周围环境的了解越来越多，在一个

反馈回路中，神经元之间的联系会变得越来越强或越来越弱。神经网络（NN）也使用这种方法来修改不同神经元层（节点）之间的连接强度。然而，NN 通常部署某种形式的训练算法，它调整节点以从源数据中提取所需的特征。就像人类一样，一个神经网络可以通过一张狗的图像进行归纳，逐渐形成鉴别不同种类的狗的能力。

在深度学习当中，人工神经元是一个包含输入、计算与输出功能的模型，与生物神经元的对应关系如图 2.1 所示。

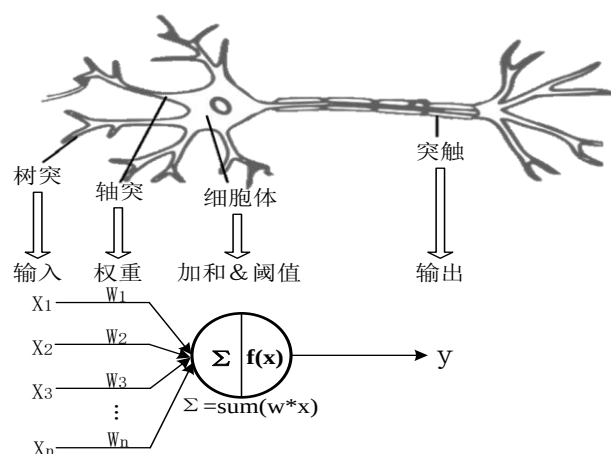


图 2.1 神经元对应关系

Figure 2.1 Neuron correspondence

2.2.2 神经网络结构

1) 神经元模型

神经网络也被称为“人工神经网络”或“并行分布式处理系统”或“连接系统”，由神经元模型构成，每个神经元通过一个被称为“权重”的链接与其他神经元进行连接，该权重具有与输入信号相关联的重要信息，这对于神经元完成特定任务来说是最有价值的信息，因为权重通常会激发或抑制正在传达的信号。每个神经元都有一个被称为“激活信号”的内部状态。将输入信号基于激活规则进行组合后产生的输出可以发送到其他神经单元。神经元模型包括输入、输出以及计算功能，一个典型的神经元模型如图 2.2 所示。

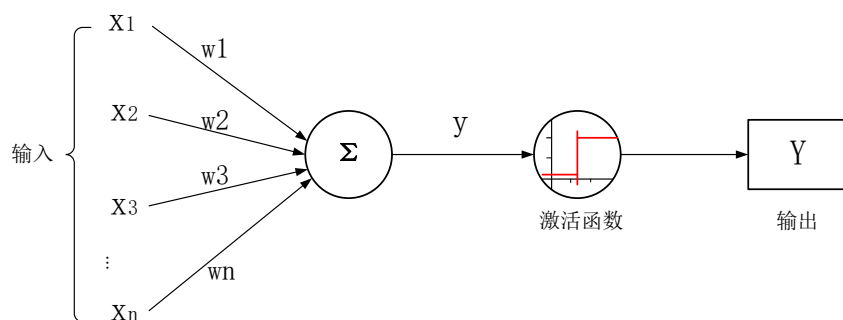


图 2.2 神经元模型

Figure 2.2 Neuron model

由以上模型可以看出，输入信号与输出信号之间形成了对应关系，神经元与关联权重的乘积后进行加和再经过激活函数得到输出 Y 的值。将输入信号记为 $(x_1, x_2, x_3, \dots, x_n)$ ，对应权重记为 $(w_1, w_2, w_3, \dots, w_n)$ ， y 表示叠加和， Y 表示经过激活函数的输出结果，则该神经元的计算可表示为式 (2.1)。

$$y = x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + \dots + x_n * w_n = \sum_i^n x_i * w_i \quad \dots (2.1)$$

输出 Y 通过在加权和 y 上应用激活函数来得到，表达式如 (2.2) 所示。

$$Y = f(y) \quad \dots (2.2)$$

其中 f 为激活函数，该函数在卷积计算中承担着非常重要的角色，通常跟在卷积层之后。在神经网络中，称为输入的数字数据点被馈送到输入层的神经元中。每个神经元都有权重，将输入数字乘以权重即可得到神经元的输出，该输出将传输到下一层。激活函数是馈送当前神经元的输入与其进入下一层的输出之间的数学“门”，根据每个神经元的输入是否与模型的预测相关来确定是否应激活（“触发”）该功能（任园园, 2020）。激活函数的引入使得神经网络可以学习更多功能。

常见的激活函数有：

① Sigmoid 函数，该函数具有平滑的渐变，采用实值输入并将其压缩为 0 到 1 之间的范围。Sigmoid 的函数图像如图 2.3 (a)所示，表达式如 (2.3) (2.3) 所示。

$$\sigma(x) = \frac{1}{1+e^{-x}} \quad \dots (2.3)$$

② ReLu 函数，相对于其它激活函数而言，该函数具有更高的计算效率，允许网络快速收敛，并且可以缓解梯度消失问题。其函数图像如图 2.3 (b)所示，如式 (2.4)。

$$f(x) = \max(0, x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases} \quad \dots (2.4)$$

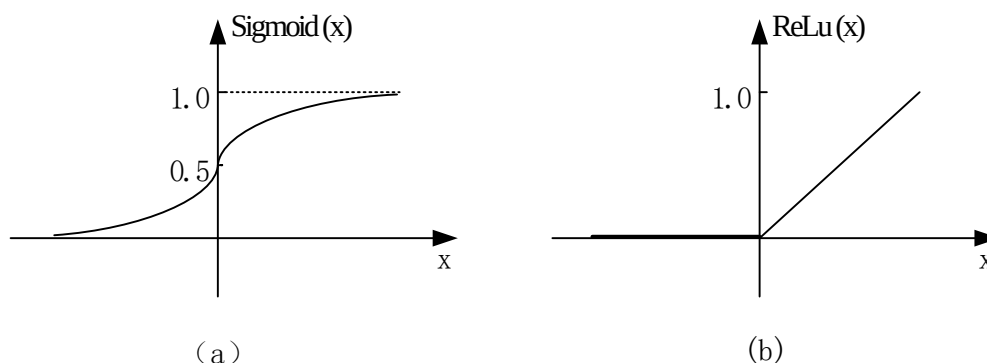


图 2.3 Sigmoid 及 ReLu 函数图像

Figure 2.3 Diagram Sigmoid & ReLu function

2) 多层感知器

感知器 (Perceptron) 由 Frank Rosenblatt 于 1957 年提出，它是用于进行某些计算以检测输入数据中的特征的神经网络单元 (人工神经元)，在二进制分类中起着重要的作用。单层感知器是最简单的人工神经网络类型，只能分析具有二进制结果 (即 1 和 0) 的线性可分离对象，不包含任何隐藏层。多层感知器的结构类似于具有更多隐藏层的单层感知器的神经网络模型。与单层感知器相比，具有更大的处理能力。简单地说，多层感知器可以被看作是一个由无数人工神经元组成的网络，它的激活函数不再是线性的，而是部署了 Sigmoid、tanh、ReLU 等非线性激活函数。分为正向和反向两个阶段执行，其中正向阶段激活函数从输入层到输出层，而在反向阶段，实际观测值与需求给定值之间的误差从输出层起向后求导，用于修改权重和偏差值。具有单个隐藏层的多层感知器 (所有连接具有与之关联的权重) 如图 2.4 所示。

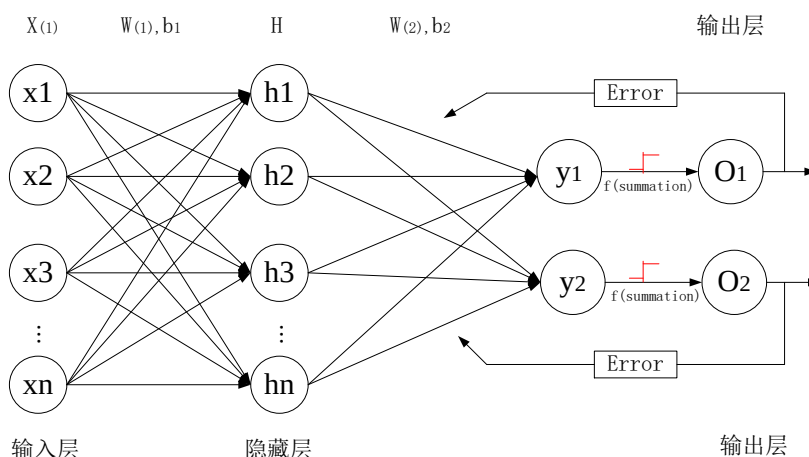


图 2.4 多层感知器

Figure 2.4 Multilayer perceptron

2.3 卷积神经网络

与人类学习识别对象的方式类似，我们需要向算法展示数以百万计的图片，然后才能对输入进行泛化，并对从未见过的图像进行识别。与人类不同的是，在计算机的世界里只有数字，每个图像都被表示为数字，称之为像素。它们以不同的方式感知图像的事实并不意味着无法像人类一样识别模式（王立有，2019）。为了教授算法如何识别图像中的对象，可以使用一种特定类型的人工神经网络：卷积神经网络（Convolutional Neural Networks, CNN）。它的名称源于网络中最重要的操作之一：卷积。

2.3.1 卷积神经网络基本结构

在视觉中，单个感觉神经元的感受区是视网膜上的一个特定区域，在这个区域内，某些东西会影响神经元的放电（也就是说，会激活神经元）。每个感觉神经元细胞都有类似的感受区，并且它们的感受区覆盖在神经元上。对于传统神经网络模型来说，假设输入一个 64×64 的彩色图像，则仅第一层的单个神经元上的权值就增加到 12288 ($64 \times 64 \times 3$)，除此之外，还需要考虑网络的大小，造成了很高的计算复杂性。因此，人们提出了卷积神经网络，通过了解物体的形状来识别模式。

CNN 是一类前馈型深度神经网络，它使用多层感知器的变化形式开发而成，

以要求尽可能少的预处理，与常规神经网络具有不同的体系结构。常规神经网络通过将输入置于一系列隐藏层中来转换输入。每一层都由一组神经元组成，其中每一层之前都与该层中的所有神经元完全连接。最后，最后一个完全连接的层（输出层）代表了预测。卷积神经网络有点不同。首先，这些图层按宽度、高度和深度 3 个维度进行组织。并且，相邻层中的所有神经元不进行全连接，仅连接部分。最后，最终输出将减少为沿着深度维度组织的概率分数的单个向量。

卷积神经网络由多个卷积层（CONV layers）和全连接层（FC layers）组合而成。在该网络卷积层中，每层产生了对输入图像的高层抽象表示的结果，称之为特征图（Feature Map, FM）。底层的特征图能够获得输入图像的细节信息（如边缘、角点、颜色等），越往顶层越能获得输入图像的整体信息（如形状、轮廓等），正是卷积神经网络利用了此分层特性，取得了很好的性能提升。

CNN 体系结构有多种变体。但是，它们的基本组成部分非常相似（Rawat, &Wang, 2020）。LeNet 是典型的卷积神经网络之一，早期用于读取邮政编码，数字等字符识别任务。CNN 学习方法和体系结构的各种改进正在不断发展，以使 CNN 可扩展到大型，异构，复杂和多类问题，有效推动了深度学习领域的发展。近年来，提出了许多新的神经网络结构，但都是对 LeNet 的改进，主要由多个卷积层（CONV layers）、池化层（Pooling layers）和全连接层（FC layers）三种类型的层组合组成。LeNet 的网络结构如图 2.5 所示。

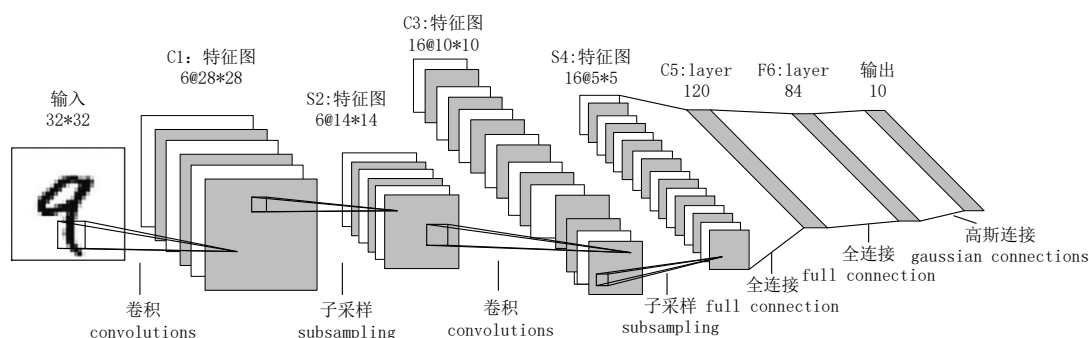


图 2.5 LeNet 的网络结构

Figure 2.5 LeNet's network structure

1) 卷积层

CNN 是从“卷积”运算符派生出来的，卷积层是卷积网络的核心构建块，它完成了大部分计算繁重的工作。在计算机中，每个图像都可以表示为一个像素值矩阵，对于 CNN，卷积意味着两个函数的数学组合以获得第三个函数，其主要目的是从输入图像中提取特征。它在滑动窗口机制上起作用，从输入特征矩阵的左上角开始，卷积核向右端滑动，当它到达输入特征矩阵的右端时，将移回到左端并移至下一行。该机制有助于从图像中提取特征，生成要素图。对于一个 5*5 的像素的图像，通过一个 3*3 大小卷积核，其卷积计算如图 2.6 所示。

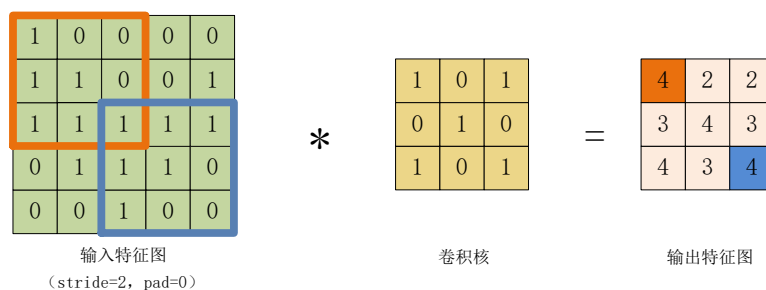


图 2.6 卷积计算

Figure 2.6 Convolution

其中 5*5 的矩阵称为输入特征图，中间 3*3 的矩阵被称为“过滤器”或“卷积核” (Convolutional kernel/ kernel) 或“特征检测器”，最右边的 3*3 矩阵称为“卷积特征”或“特征图 (Feature map, FM)”。卷积计算为：每次取出卷积核矩阵，将其对输入矩阵依次扫描并进行内积的运算过程，当 kernel 扫描完成后就得到整个卷积计算的结果。计算表达式如 (2.5) 所示。

$$output_{b,o_c,x,y} = \sum_{i_c=0}^{I_c-1} \sum_{i=0}^{F_h-1} \sum_{j=0}^{F_w-1} input_{b,i_c,x+i,y+j} \times filter_{o_c,i_c,i,j_c} \quad \dots (2.5)$$

其中 F_h 是卷积核的高度， F_w 是卷积核的宽度， i 是卷积核中行的索引， j 是卷积核中列的索引， x 和 y 分别是输入特征图中行的索引及列的索引。其中 $0 \leq b < B, 0 \leq o_c < O_c, 0 \leq x < O_h, 0 \leq y < O_w$ (O_h : 输出特征图的高度; O_w : 输出要素图的宽度)。从上面的计算中可以明显看出，将相同的输入与不同的卷积核进行计算，可产生不同的输出，因此只需在卷积运算之前采用不同的卷积矩阵即可从图像中检测不同的特征，如边缘、曲线等。

2) 池化层

池化层通常直接在卷积层之后，用于简化来自上一层的信息。这样，就像在卷积中一样，选择一个面积单位（例如 2×2 ）以遍历整个上一层的输出。该单元负责将来自该区域的信息汇总为唯一值。如果上一层的输出为 24×24 ，则池化后得输出将为 12×12 。池化操作对激活图进行下采样，通常在两个维度上均降低两倍，目的是减少学习权重的数量，并避免过度拟合。除此之外，池化的方式也很重要，常见的方法有最大池化和平均池化，其中最大池化（Max pooling）是最常使用的方法。该方法通过取其最大值来合并相邻 2×2 单元中的像素，保留特征最大值，从而提高输入特征的鲁棒性。池化的优势在于，它使神经网络计算能够压缩数据量而又不会丢失太多信息，并且为原始图像中的平移产生了一定的不变性。由于没有权重或参数需要学习，该操作需要的成本较低。图 2.7 显示了最大池化的一个示例。

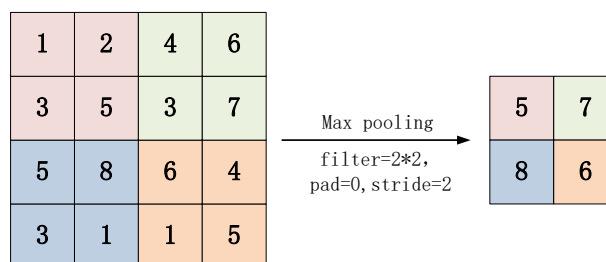


图 2.7 最大池化示例

Figure 2.7 Max pooling example

3) 全连接层

“完全连接”表示相邻层中的每个神经元都互相连接。一个完全连接的层放在网络的末端，其输入是前一层的输出，其输出是 N 个神经元，其中 N 是模型中用于最终确定分类的类的数量。完全连接层是传统的多层感知器，在输出层中使用 Softmax 激活功能或其它分类器（如 SVM）。卷积层和池化层的输出表示输入图像的高级特征，全连接层的目的是利用这些特征根据训练数据集将输入图像分类成不同的类。除了分类之外，添加一个完全连接的层通常也是学习这些特征的非线性组合的一种廉价方法（刘明, 2020）。全连接层输出概率之和为 1，这是通过在全连接层的输出层使用 Softmax 作为激活函数来保证的。Softmax 函数取一个由任意实值分数组成的向量，并将其压缩为一个由 0 到 1 之

间的和为 1 的值组成的向量。

2.3.2 卷积神经网络的特性

1) 局部连接

传统的神经网络在相邻的两层之间，所有的神经元相互连接，这种方式被称为全连接。局部连接类似人眼识别一个物体的过程当中，并不是一眼就能全部看清，而是通过对局部观察进行拼接从而得到整体的识别。这体现了特征只与局部输入有关的特性，卷积神经网络利用该特性来减少训练的参数量。局部连接指每个神经只连接到输入图像子集，相邻两层的神经元每一层的仅与其前一层的神元进行部分连接，从而使得卷积核大小小于输入图像的大小，有效减少了神经元直接连接数目（何鹏程, 2015）。

2) 参数共享

参数共享是指在计算相同的输出特征图时，使用相同的参数值计算每个输出像素，即使用相同的卷积核遍历整个输入图像。其最广泛的应用是在卷积神经网络中。自然图像有许多对平移不变的统计属性，例如将猫的照片向右平移一个像素，它仍然是猫的照片，即特征与其在输入中的位置无关。CNN 通过在多个图像位置共享参数来考虑这个属性，这样，无论猫出现在第 i 列还是第 $i+1$ 列，我们都可以用相同的 `cat` 检测器找到一只猫。参数共享基于这样的假设，即如果一个区域特征对于在一个设定的空间区域进行计算是有用的，那么它可能在另一个区域也是有用的。如果我们将输出体积内的每个单独的激活图限制为相同的权重和偏差，那么我们将看到卷积层产生的参数数量的大幅减少。参数共享意味着不必在每个单独的位置学习特定的卷积内核，而是可以在所有位置使用相同的卷积内核。这种方式有效地减少了卷积神经网络系统的存储空间，提高了计算效率。

3) 卷积输出特征图的大小由执行卷积步骤之前需要确定的三个参数：

深度（Depth）：深度对应于我们用于卷积运算的卷积核数量。利用不同深度的滤波器对原始数字图像进行卷积，可以产生多个不同的特征图。

步幅（Stride）：步幅是我们在输入矩阵上滑动卷积核矩阵的像素数。步幅较大时将产生较小的特征图。

零填充 (Zero-padding): 零填充可以方便地控制特征图通过卷积后的输出大小。

2.4 卷积计算加速技术研究

深度学习 (DL) 领域发展迅速, 该领域的进步令人震惊并且不懈。如今, 深度学习领域将大不相同, 能够直接在边缘设备 (例如电话, 智能扬声器, 摄像头, 车辆) 上运行 AI 算法具有巨大优势, 处于边缘的应用和研究将成为主流。但是, 要使边缘智能无处不在的崇高愿景成为现实, 就需要一项关键的技术突破: 神经网络模型需要变得更小。因此, 在不损害神经网络性能的情况下开发神经网络的技术已成为深度学习领域最重要的追求之一。如今, 典型的深度学习模型非常庞大, 需要大量的计算和存储资源才能运行, 然而边缘设备中的硬件处理器不足以支持它们。因此, 开发使神经网络和框架模型更轻量级的方法称为关键: 它将释放神经网络构建的一系列产品和业务机会。

2.4.1 基于模型的优化方法

近年来, 研究人员在神经网络模型优化方面取得了长足的进步, 开发了一系列使神经网络小型化的技术。这些技术可以分为五个主要类别: 修剪、量化、低秩分解、紧凑卷积滤波器和知识蒸馏。一般来说, 参数剪枝和共享、低秩因子分解和知识蒸馏方法可以用于具有全连接层和卷积层的数据网络, 实现相当的性能。另一方面, 使用转移/紧凑过滤器的方法是为具有卷积计算特征的模型设计的。低秩因子分解和基于转移/紧凑滤波器的方法提供了端到端的流水线, 并且可以容易地在 CPU/ GPU 环境中实现。

1) 网络剪枝

现有的研究表明, 我们经常可以从训练有素的神经网络中删减 90% 或更多的参数, 而不会损害它们成功完成训练它们的任务的能力。这产生了网络剪枝的想法: 在神经网络中的许多参数中, 有部分是多余的、对输出的贡献不大的参数, 所以针对已训练好的深度神经网络模型, 我们通过移除冗余的、信息量少的权值, 就可以减少网络模型的参数, 进而加速计算以及压缩模型的存储空间, 同时还能防止模型过拟合 (纪荣嵘 等, 2018)。(Wang et al., 2019) 提出

了允许从头开始修剪的网络修剪管道，该管道减少了使用常规修剪方法时发生的预训练开销，并且还提高了网络的准确性。（Madaan et al., 2019）考虑了深度神经网络中潜在特征的脆弱性，所提出的方法在保留健壮特征的同时修剪了脆弱的特征。（Chen et al., 2019）提出通过自适应网络修剪方法（SANP）减少 CNN 的计算成本。该方法通过为每个卷积层引入显着性和修剪模块（SPM）来实现。剪枝存在的缺点是，只有在训练模型后，它才会减少使用模型的成本，并且如果我们一次修剪太多，可能会损坏网络，导致无法恢复。

2) 参数量化

参数量化通过减少表示每个权重所需的位数来压缩原始网络。权重可以量化为两个值（二进制），三个值（三进制）或多位。它可以是统一的，也可以是非统一的。（Han S, 2016）提出的方法首先修剪不重要的连接，然后重新训练稀疏连接的网络。然后利用权值共享对链路权值进行量化，并对量化后的权值和码本进行霍夫曼编码，进一步降低码率。这项工作在所有基于参数量化的方法中实现了最先进的性能。有很多直接用二进制权值训练 CNNs 的工作，例如二进制网络（Courbariaux and Bengio, 2016）和异或非网络（Rastegari et al., 2016）主要思想是在模型训练过程中直接学习二元权重或激活。（Merolla et al., 2016）提出的系统研究表明，用反向传播训练的网络可以抵抗特定的权重失真，包括二进制权重。这些方法或者在训练之后或者甚至在训练期间量化或者二值化网络权重，其中参数之间的重要性或者冗余被完全忽略。尽管实现了高压缩率，但这些方法的准确性在压缩后会急剧下降。

3) 紧凑网络设计

使用紧凑的深度网络可以直接减少相关的计算成本。近年来，已经提出了几种方法来遵循这一思路。实现此目标的一般思路包括使用小型卷积核，分组卷积和高级正则化，以降低存储和计算复杂度，在保持可比精度的同时实现整体加速。许多最近的 CNN 架构在构建轻量级模型时遵循了紧凑的网络设计策略。

（Min Lin, 2014）提出的 NIN 架构网络，主要使用了 1×1 卷积来增加网络容量，同时使得计算量和空间复杂度的降低。这种思想在 GoogleNet 和 ResNet 等著名模型也得到了广泛地使用。ResNet 通过使用组卷积，在与 ResNet 计算成本开销

相同的情况下，获得了更高的精度。MobileNet 模型应用了深度卷积来降低计算成本和存储开销，与 VGG-16 相比，该模型的大小缩小了 32 倍，并且加快了 27 倍的速度。ShuffleNet 引入了信道混洗操作，以增加多个组内的信息变化，与 AlexNet 相比，在精度相当的情况下速度提高了约 13 倍。

2.4.2 基于计算框架的优化方法

除了对神经网络模型进行优化，对神经网络计算的加速还可以采用加快框架执行时间的方法，这种方法主要进行的是卷积层及全连接层卷积计算中的矩阵运算优化，旨在减少推理过程中发生的操作次数，不会影响模型的参数。对框架的优化方法主要有三种变换。im2col 方法对特征数组和权重数组进行了重构，将深度卷积转化为矩阵乘法。FFT 方法在频域上操作，将卷积转化为乘法。在 Winograd 滤波中，卷积归结为基于元素的矩阵乘法，这归功于数据的平铺和线性转换。这些计算转换主要出现在时间架构中，并通过各种线性代数库实现，如针对 CPU 的 OpenBLAS 或针对 GPU 的 cuBLAS。

1) im2col+gemm

在 CPU 或 GPU 中，处理 CNN 的常用方法是使用 im2col 将卷积层和全连接层映射为一般矩阵乘法 GEMM。im2col 方法将输入特征映射以及给定卷积层的所有权值重新排列为一个矩阵，使每个特征映射被压缩成为一列，然后通过两个矩阵相乘来计算卷积。（Suda et al., 2016）以及（Zhang and Jing, 2017）利用 im2col 为 CNN 开发基于 OpenCL 的 FPGA 加速器，然而，这种方法引入了大量冗余数据，可能会导致存储效率低下或复杂的内存访问模式。

2) FFT

快速傅里叶变换快速傅里叶变换（FFT）是一种著名的将二维卷积变换为点对点乘法算法。使用 FFT 处理 2D 卷积可将复杂度从 $O(w^2 \times k^2)$ 降低到 $O(w^2 \log^2(w))$ ，这被用于开发基于 FPGA 的加速器和 CNN 推理（Ko et al., 2017）。与标准卷积和 Winograd 算法相比，FFT 比较擅长于大尺寸卷积核的卷积计算（Bottleson et al., 2016）。使用重叠加法（Highlander and Rodriguez, 2016），可以将 FFT 卷积的计算复杂度进一步降低到 $O(W \log_2(k))$ ，这种方法可以应用于输入大小远远大于卷积核大小的情况下。

3) Winograd 变换

Winograd 最小滤波算法是一种计算变换, 通过使用加减运算替代部分乘法运算达到加速效果, 可以应用于处理步数为 1 的卷积, 这在 CNN 中是很常见的。尤其在处理小卷积时, 该算法非常有效。(Dicecco et al., 2017) 中研究了 Winograd 滤波在基于 FPGA 的 CNN 加速器中的应用, 并在执行 AlexNet 卷积层时产生了 46 GOPs 的计算吞吐量。(Aydonat et al., 2017) 优化数据路径以支持 Winograd 卷积(通过使用循环展开和平铺策略), 并将中间 FM 存储在片上缓冲区, 性能提高了 42 倍。(Lu et al., 2017) 使用同样的方法在 Xilinx ZCU102 设备上推出 CNN 加速器, 在 VGG 卷积层上产生了 2.94 TOPs 的吞吐量。

4) ARM NEON

NEON 技术是 ARM Cortex-A 系列处理器的 128 位 SIMD(单指令, 多数据)架构扩展, 专门针对大规模并行运算设计, 旨在为消费性多媒体应用程序提供灵活、强大的加速功能, 从而显著改善用户体验(王韬, 2017)。对于基于 ARM 架构的处理器而言, 可以使用 NEON 优化标记编译。在 32 位 Cortex-A 系列处理器上, 运行大量 8 位或 16 位操作的效率相对较低。处理器 ALU、寄存器和数据路径是为 32 位计算而设计的。如果它们用于 8/16 位操作, 则需要额外的指令来处理溢出。SIMD 允许单个指令将寄存器值视为多个数据元素, 并对这些元素执行多个相同的操作, 进行加速运算。ZYNQ-7000 平台集成了 NEON, 可以使用 NEON 优化库、编译器自动矢量化、NEON 内联函数以及汇编指令进行计算的并行优化。

2.5 本章小结

本章节首先介绍了神经网络的基本概念和结构, 接着对深度学习算法中得到广泛使用的卷积神经网络算法进行了详细说明, 最后对当前的神经网络加速方法进行了详细阐述。为后面的设计奠定理论基础。最后总结出, 使用深度可分离卷积紧致神经网络模型设计, 利用加速算法对框架中的卷积计算进行加速, 可在一定程度上降低计算的参数量, 有效提升计算速度, 为神经网络技术应用到嵌入式平台提供了解决方案。

第3章 基于 ZYNQ Linux 的图像采集模块构建

3.1 绪论

由于 Linux 开源、低成本、方便使用等优势，有很多基于嵌入式 Linux 内核的系统，如智能手机、平板电脑、掌上电脑、智能电视、机器控制和医疗仪器等。嵌入式系统的设计开发分为裸机开发和基于嵌入式操作系统开发两种模式。前者主要是面向硬件开发，用户通过地址直接对物理硬件实现操作，开发难度较大。后者为面向操作系统开发，通过操作系统对软硬件资源进行管理，使得嵌入式系统设计更加灵活方便，当系统功能复杂时，可有效降低开发难度，提高开发效率。本研究选择面向操作系统的开发方式，构建 ZYNQ SoC 的 Linux 操作系统，设计并实现了图像采集输入模块，使得面向 ZYNQ SoC 的神经网络计算框架可以进行实际应用。Linux 操作系统的构建分为硬件配置、软件编译和系统移植三个部分。

3.2 ZYNQ SoC 硬件平台介绍

ZYNQ-7000 系列开发平台是基于 Xilinx 公司生产的全可编程片上系统（All Programmable System On a Chip, AP SoC）开发板，集成了 ARM Cortex-A9 双核处理系统（Processing System, PS）和传统 FPGA 可编程逻辑（Programmable Logic, PL）单元结构，具有高度的灵活性和可扩展性（聂阳 等, 2020）。片上系统（SoC）是一种集成电路，它将计算机或其他电子系统的所有组件集成到单个芯片中。ZYNQ 是一个完整的片上系统，将 PL 和 PS 两大功能模块完美结合，支持广泛的特定接口功能，如千兆收发器、高性能 I/O、高吞吐量 AXI（高级扩展接口）、数千个 PS 至 PL 连接等。PL 部分为传统 FPGA，PS 部分包含了双 ARM Cortex-A9 MPCore 处理器，还包括片内存储器、外部存储器接口和一套丰富的 I/O 外设可以运行 Linux 等操作系统，实现多任务的处理、图像等相关数据的存储管理、中断处理等功能。这种双芯片组合的全可编程片上系统降低了系统的成本、复杂性、尺寸和功耗，同时还能提高系统性能。这种基于 ARM

的 PS 和片上 PL 之间的紧密集成为设计人员添加几乎任何外设或创建可以扩展系统性能的自定义加速器创造了无限的可能性。

ARM 是各式各样的智能设备，如智能手机、智能电视、平板电脑中最常见的处理器，成为对开发者有吸引力的领域。ZYNQ-7000 具有 ARM Cortex-A9 双核，使得处理器在任何情况下都不会成为 ZYNQ 板上开发应用的瓶颈。此外，ZYNQ-7000 还包括 512MB DDR3 存储器、256MB 闪存和 SD 插槽，提供了允许小尺寸系统存储在闪存中的灵活性，并允许将大型系统存储在外部 SD 卡中。因此，可以在 ZYNQ 平台开发一个快速或者大型的嵌入式 Linux，实现更加灵活地设计和开发。

3.3 硬件环境搭建

基于 SoC 的嵌入式 Linux 系统移植方法是修改和移植 Bootloader 引导程序 U-boot、编译 Linux 内核、生成内核镜像、制作文件系统，然后烧写到开发板上进行启动（张朝元 等, 2018）。在 ZYNQ-7000 系列开发板上移植 Linux 操作系统相对复杂一点，硬件环境作为 Linux 系统运行的基础，主要完成 PS 部分的硬件配置和针对特定应用的 PL 逻辑的开发。

根据 Linux 启动要求，ZYNQ 7020 嵌入式平台的 Linux 操作系统构建需要的最小硬件配置如下：

- 1) 一个三时序计数器（TTC）（必要）
- 2) 外部存储器至少有 32 MB 内存（必要）
- 3) 串行控制台 UART（必要）
- 4) 非易失性存储器（可选），如 QSPI 闪存和 SD/MMC
- 5) 以太网（可选，对网络访问必不可少）

所需最小硬件配置如图 3.1 所示，搭建好硬件平台后，最终可生成硬件配置的二进制文件 `system_wrapper.bit` 以及 `fsbl.elf` 启动文件。

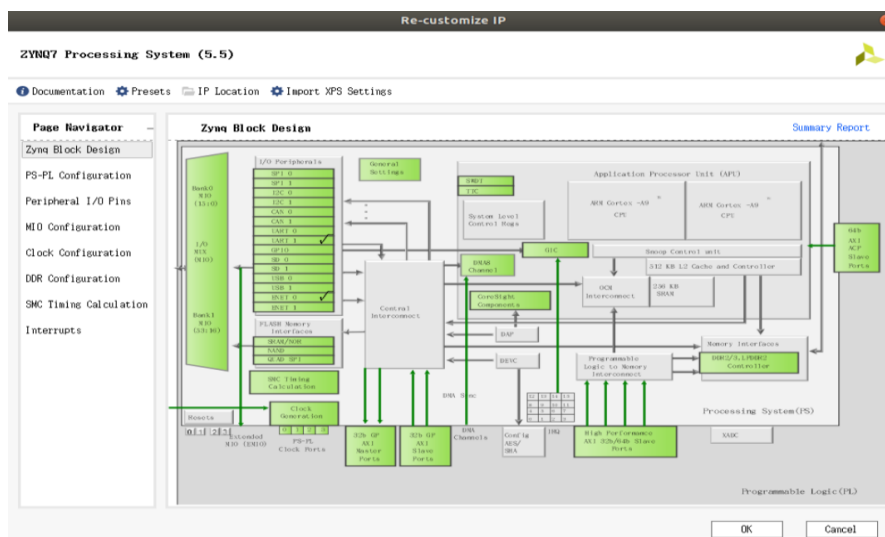


图 3.1 硬件配置

Figure 3.1 Hardware configuration

3.4 软件环境开发

软件环境开发是 Linux 系统移植的核心，如图 3.2 所示，主要包括编译环境配置以及 U-boot、Linux 内核（Kernel）、设备树（Devicetree）和文件系统（Rootfs）四个部分的编译。其中软件环境搭建为后面编译操作系统软件提供编译环境，保证 Linux 的顺利构建。U-Boot 是在操作系统之前执行的一小段程序，主要负责对设备进行简单初始化；Linux 内核 Kernel 是操作系统的核心，主要负责系统的进程管理、内存管理、文件系统、网络功能；设备树 Devicetree 以树的形式对 ZYNQ 相连的硬件设备进行描述，使内核的设计和编译脱离对硬件的依赖；文件系统 rootfs 为操作系统提供在存储设备上存储数据的方法。

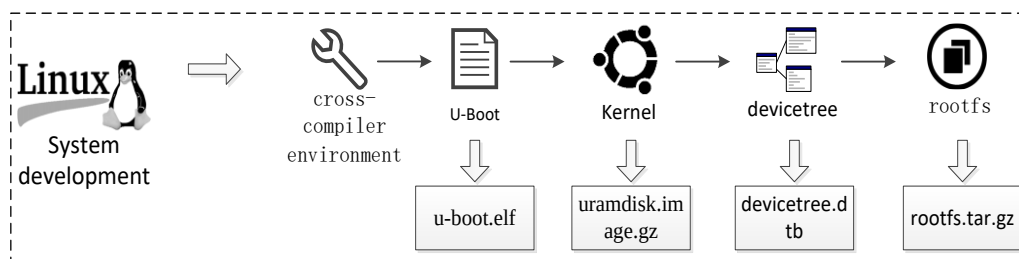


图 3.2 软件开发流程

Figure 3.2 Software development

本文采用 SD 卡来加载操作系统，总共需要以下四个文件来启动 Linux：

1) 引导映像 (boot.bin): 该文件是一个二进制合成映像, 由不同的文件组成。最简单的 Linux 系统在引导映像中只需要三个组件: 第一阶段引导加载程序 FSBL (fsbl.els)、可编程逻辑硬件比特流 (system.bit) 和第二阶段引导加载程序 U-Boot (u-boot.elf)。

2) 设备树二进制文件 (devicetree.dtb): 从设备树源文件获得, 并通过 U-Boot 加载到 DDR 内存中。内核必须知道它正在工作的硬件的每一个细节, 为此, 它使用称为“设备树二进制”或“设备树二进制 (DTB)”的数据结构来描述硬件。

3) Linux 内核文件 (uImage): 也通过 U-Boot 加载到 DDR 内存中, 它初始化系统硬件并装载根文件系统。

4) 根文件系统映像 (uramdisk.image.gz): 同样是通过 U-Boot 加载到 DDR 内存中, 它包含操作系统本身。

3.4.1 编译环境搭建

交叉编译是使开发者能够编译应该在“目标”体系结构上运行的代码, 并在不同(“主机”)体系结构上工作的过程中进行编译的过程。在“通常”的构建过程中, “主机”和“目标”平台是相同的。例如, 使用 PC 编译代码以使其在同一 PC 上运行。当“主机”和“目标”平台不同的时候, 需要使用交叉编译。如, 使用 PC 来编译要在嵌入式设备上运行的程序。在为 ZYNQ SoC 构建 Linux 操作系统之前, 需要一个在主机上运行的针对 ZYNQ SoC 的交叉编译器来创建可执行代码。工具链可以通过许多不同的方式来实现, 本文选择最简单的方法, 下载交叉编译器 arm-linux-gnueabi 的软件包解压后在主机系统上进行安装, 并将交叉编译工具的路径添加到系统环境变量当中, 添加完成后键入以下命令 arm-linux-gnueabi-gcc -v 查看是否安装成功。

3.4.2 U-Boot 配置

U-Boot 是一个 GNU 通用引导加载器, 常用于嵌入式 Linux 设备。它是由马格努斯·达姆在 1999 年为沃尔夫冈·登克公司开发的, 类似于个人电脑的引导程序。U-Boot 是嵌入式 Linux OS 可用的最丰富、最灵活、开发最活跃的开源引

导加载程序。它可用于许多不同的计算机体系结构，不仅是 ZYNQ-7000 的 ARM 体系结构，还有其他体系结构，如 AVR32、Blackfin、MicroBlaze、MIPS 和 x86 等。UBoot 的开发与 Linux 息息相关，部分源代码源自 Linux 源码树，有一些头文件是共通的。Xilinx 提供了一个官方的 Xilinx U-Boot 存储库：u-boot-xlnx，其中包括在 Xilinx 板上运行的 U-Boot。最常见的引导启动过程为：

- 1) 加载所需的参数；
- 2) 加载 u-boot 参数的环境变量；
- 3) 加载嵌入式 Linux 操作系统内核。

U-Boot 被用作第二阶段引导加载程序，执行重要的引导功能：初始化加载内核所需的平台硬件、将 Linux 内核从引导介质加载到主内存（DDR3）、用指定的引导参数启动 Linux 内核。除了加载内核外，U-Boot 还将设备树传递给 Linux。ZYNQ SoC 配置 U-Boot 的步骤为：

- 1) 下载相关文件：`git clone https://github.com/Xilinx/u-boot-xlnx.git`
- 2) 进入文件夹：`cd u-boot-xlnx`
- 3) 设置交叉编译工具并将编译目标平台设置为 arm：

```
export CROSS_COMPILE=arm-linux-gnueabihf-
```

```
export ARCH=arm
```

- 4) U-boot 编译

```
make distclean
```

```
make zynq_zc702_defconfig
```

```
export DEVICE_TREE="zynq-zc702"
```

```
make
```

- 5) 生成 u-boot.elf:

```
copy u-boot u-boot.elf
```

3.4.3 Linux kernel

内核（kernel）是与设备中的硬件接口的最底层的易替换软件。它负责将所有以“用户模式”运行的应用程序连接到物理硬件，并允许进程（称为服务器）使用进程间通信（IPC）从彼此获取信息。Linux kernel 是一个开源的类似 Unix

的操作系统内核，由基于它的各种操作系统使用，通常是 Linux 发行版的形式。Linux 内核是自由开源软件的一个突出例子，最初由芬兰计算机科学学生 Linus Torvalds 在 1991 年构思和创建的，并迅速积累了无数的开发人员和用户，他们从其他自由软件项目中改编代码。因此，它是由世界各地的贡献者开发的，允许不断更新到当前的技术。内核编译的过程为：

1) 下载相关文件：`git clone https://github.com/Xilinx/linux-xlnx.git`

2) 进入以下目录，并打开 `xilinx_zynq_defconfig` 文件，保留 SD 卡，串口等有用信息，删除多余的接口：

```
cd linux-xlnx-xilinx-v2017.4/arch/arm/configs\
```

3) 键入以下命令加载 ZYNQ 内核配置，生成 Makefile 文件：

```
make ARCH=arm
```

```
CROSS_COMPILE=arm-linux-gnueabi- xilinx_zynq_defconfig
```

4) 编译内核，生成 zImage：

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi-
```

3.4.4 设备树文件

Linux 内核是一个运行在硬件上的嵌入式独立软件。它为使用者提供了一个标准化的接口，可以在不知道细节的情况下利用所有的硬件资源。在 Linux 中，使用一个称为设备树二进制（DTB）的数据结构来描述硬件，设备树二进制是一个数据库，代表了给定板上的硬件组件，并且是系统将低级硬件信息从引导加载程序传递到内核的默认机制。DTB 由设备树源（Devicetree Source, DTS）文件编译生成。linux-xlnx 库直接提供几乎所有可用的板/架构的 DTS 文件，在已经下载好的 linux-xlnx 库中，本文所需的文件路径为：

```
linux-xlnx-xilinx-v2017.4/arch/arm/boot/dts/zynq-7000.dtsi
```

所需的 DTS 文件可用，然而，有必要将这个人类可读的文件转换成一个合适的二进制机器可读文件，以便 U-Boot 和 Linux 能够理解它。可以使用设备树编译器（dtc）进行转换，进入 linux-xlnx-xilinx-v2017.4 文件夹，键入以下转换命令生成 devicetree.dtb：

```
scripts/dtc/dtc -I dts -O dtb -o /devicetree.dtb arch/arm/boot/dts/zynq-7000.dtsi
```


3.4.5 文件系统

根文件系统映像，是一个包含所有操作系统文件的压缩文件。它由不同的文件夹和文件组成，与安装在任何个人电脑或笔记本电脑上的普通 Linux 操作系统相同。然而，它只包含必要的文件夹，这些文件夹将用于为其设计的特定应用程序。最重要的文件夹通常会出现任何定制和嵌入式 Linux 操作系统中，解释如下：

“dev”包含设备驱动程序（用于连接外围设备）。

“lib”和“lib32”分别包含 64 位和 32 位的系统库（类似于 Windows 中的“C:\Windows\System”）。

“mnt”是其他文件系统的挂载点。Linux 只允许一个根文件系统，但是其他磁盘可以通过将它们装载到根文件系统的目录中来添加。类似于 Windows 下映射一个驱动器。

“root”和“home”分别是超级用户文件和其他用户的存储文件夹（类似于 Windows 中的“我的文档”）

“sys”和“proc”是虚拟文件系统位置，将内核参数公开为文件（类似于 Windows 注册表）。

“usr”是用户二进制文件的存储文件夹，所有的 Linux 系统程序都存储在这里（类似于 Windows 中的“程序文件”）。

此外，还有一个重要文件，“init”。它是内核启动的第一个用户空间程序，负责启动用户空间服务和程序。

本文采用 Xilinx 提供的文件系统映像 arm_ramdisk.image 来生成文件系统 uramdisk.image.gz。

3.5 系统移植

创建 BOOT.BIN：在 SDK 中选择 Xilinx Tools 菜单栏下的工具 Create Zynq Boot Image，如图 3.3 所示，依次添加已生成的 fbsl.elf、硬件配置二进制文件 system_wrapper.bit 和编译后的 u-boot 文件 u-boot.elf，创建 BOOT.BIN 镜像。

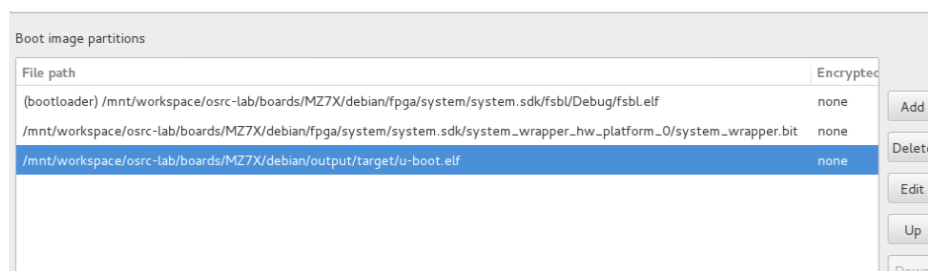


图 3.3 BOOT.BIN 创建

Figure 3.3 BOOT. BIN creation

将生成的 BOOT.BIN 和编译后的 linux 内核 uImage、设备树文件 devicetree.dtb 拷贝至 SD 卡 FAT32 格式文件夹,文件系统 rootfs.tar.gz 解压至 SD 卡 EXT4 格式文件夹。开发板设置成 SD 卡启动模式,插入 SD 卡。将 ZYNQ-7000 平台通过串口与主机连接,在主机端使用 putty 软件远程控制实现对开发板的控制,上电启动开发板后,系统启动过程如图 3.4 所示。

```
[ OK ] Started Permit User Sessions.
[ OK ] Started Serial Getty on ttyPS0.
Starting Set console scheme...
[ OK ] Started Set console scheme.
[ OK ] Created slice system-getty.slice.
[ OK ] Started Getty on tty1.
[ OK ] Reached target Login Prompts.
[ OK ] Started LSB: IPv4 DHCP client with IPv4LL support.
[ OK ] Started OpenBSD Secure Shell server.
[ OK ] Started Dispatcher daemon for systemd-networkd.
[ OK ] Reached target Multi-User System.
[ OK ] Reached target Graphical Interface.
Starting Update UTMP about System Runlevel Changes...
[ OK ] Started Update UTMP about System Runlevel Changes.

Ubuntu 18.04.1 LTS bionic-armhf ttyPS0

bionic-armhf login: █
```

图 3.4 系统启动界面

Figure 3.4 System startup interface

3.6 OV5640 图像采集模块

为了使本文设计的神经网络计算框架能够进行测试和在真实的工业环境中进行使用,本节设计并实现了 OV5640 摄像头图像采集功能模块,该模块的图像采集架构如图 3.5 所示。摄像头采样接口采集到的图像数据,通过视频输入核 Vid in 后,与 VDMA 进行数据通信,实现将采集到的图像数据通过 VDMA 写入到 DDR 中。每次写入一副图像的数据后,会把数据进行三缓存,最后再通过

VDMA 写入 SD 卡。

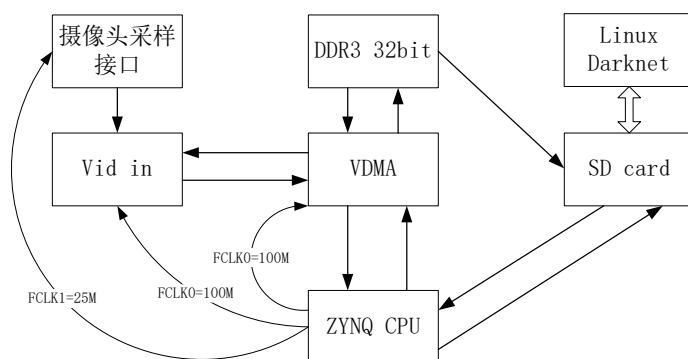


图 3.5 图像采集模块架构

Figure 3.5 Image acquisition module architecture

3.6.1 VDMA

VDMA (Video Direct Memory Access) 是 Xilinx 提供的一个 IP 核，可以完美契合 ZYNQ 的内部架构，方便地实现双缓冲和多缓冲机制，有效缩短开发周期。该 IP 能够高效地实现数据存取，在基于 ZYNQ 的图像以及视频处理系统中得到了广泛使用。

VDMA 实质上是一个搬运数据的 IP，为 DDR3 的数据读写提供了一种便捷的方案。其数据接口分为读和写两个通道。写通道可以把 AXI-Stream 类型的数据流写进 DDR3 中，读通道可以从 DDR3 中读取数据，并且以 AXI-Stream 类型输出。通过将数据存储到 DDR3，CPU 就可以方便地对数据进行处理和解析，例如对图片进行缩放、裁剪等操作，从而就可以在神经网络中对采集的图像进行识别或检测。AXI VDMA 操作从视频参数和起始地址寄存器以及 VDMA 控制寄存器的设置开始。控制初始化操作如下。

1) 首先将控制信息写入通道 VDMACR 寄存器 (MM2S 的偏移量 0x00, S2MM 的偏移量 0x30) 以设置中断允许，并设置 VDMACR.RS = 1 以启动 AXI VDMA 通道运行。

2) 将有效的视频帧缓冲区起始地址写入通道 START_ADDRESS 寄存器 1 至 N，其中 N 等于帧缓冲区 (MM2S 的偏移量 0x5C 最高为 0x98, S2MM 的偏移量 0xAC 最高为 0xE8)。当 AXI VDMA 配置为大于 32 的地址空间时，每个起

始地址将被编程为两个寄存器的组合，其中第一个寄存器用于指定地址的 LSB 32 位，而下一个寄存器用于指定 MSB 32 位。

3) 写入有效的帧延迟并跨步至通道 FRMDLY_STRIDE 寄存器（对于 MM2S，偏移量 0x58；对于 S2MM，偏移量 0xA8）。

4) 将有效的水平大小写入通道 HSIZE 寄存器（对于 MM2S，偏移量为 0x54，对于 S2MM，偏移量为 0xA4）。

5) 将有效的垂直大小写入通道 VSIZE 寄存器（对于 MM2S，偏移量 0x50，对于 S2MM，偏移量 0xA0）。这将启动频道传输视频数据。可以在运行时随时通过 AXI4-Lite 控制界面编写新的视频参数和视频起始地址来更新视频参数设置。在为各个通道写入垂直大小寄存器后，新写入的视频传输值将在下一帧边界生效。

3.6.2 FPGA 工程配置

OV5640 图像采集单元底层设计如图 3.6 所示。

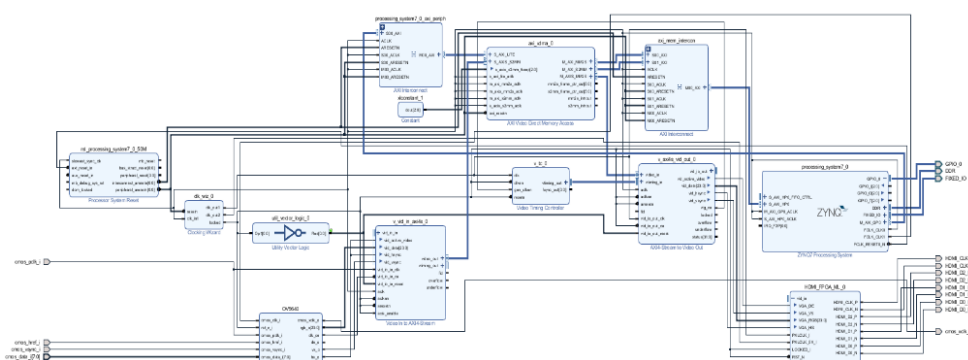


图 3.6 图像采集底层设计

Figure 3.6 Underlying design of image capture unit

图像采集输入使用了两个 IP，一个是 OV5640 自定义 IP，实现将摄像头采集的 RGB565 数据格式转换为 RGB888 格式输出；另外一个 Video In to AXI-Stream 官方提供的 IP，实现将 RGB 数据转换为 AXI-Stream，输出给 VDMA 进行处理。图像处理使用了 VDMA IP，在 VDMA 中设置三帧缓存区，从而实现在两个不同的时钟域之间传递图像帧而不会产生剪切，将采集的视频很好的缓存在三帧缓冲区中而不会发生撕裂现象。通过 VDMA 与 DDR 交互，能够很好

的进行图像数据的读写。最后从 DDR 中读取一帧图像保存在 SD 卡中。主要实现代码如下所示。

```
Miz702_EMIO_init();//初始化 EMIO 寄存器
ov5640_init_rgb(0);//初始化 OV5640 寄存器
//VDMA 向 DDR 中写数据通道初始化
Xil_Out32((VDMA_BASEADDR + 0x030), 0x108B);//使能循环写模式
Xil_Out32((VDMA_BASEADDR + 0x0AC), VIDEO_BASEADDR0);//三帧缓冲区 0 的起始地址
Xil_Out32((VDMA_BASEADDR + 0x0B0), VIDEO_BASEADDR1);// //三帧缓冲区 1 的起始地址
Xil_Out32((VDMA_BASEADDR + 0x0B4), VIDEO_BASEADDR2);// //三帧缓冲区 2 的起始地址
Xil_Out32((VDMA_BASEADDR + 0x0A8), (H_STRIDE*3));//水平方向上的跨度
Xil_Out32((VDMA_BASEADDR + 0x0A4), (H_ACTIVE*3));//水平方向数据总量
Xil_Out32((VDMA_BASEADDR + 0x0A0), V_ACTIVE);//竖直方向总共有多少行
//从 DDR 中读取数据设置
Xil_Out32((VDMA_BASEADDR + 0x000), 0x8B); //使能循环读模式
Xil_Out32((VDMA_BASEADDR + 0x05c), VIDEO_BASEADDR0);
Xil_Out32((VDMA_BASEADDR + 0x060), VIDEO_BASEADDR1);
Xil_Out32((VDMA_BASEADDR + 0x064), VIDEO_BASEADDR2);
Xil_Out32((VDMA_BASEADDR + 0x058), (H_STRIDE*3));
Xil_Out32((VDMA_BASEADDR + 0x054), (H_ACTIVE*3));
Xil_Out32((VDMA_BASEADDR + 0x050), V_ACTIVE);
while (1) {
    sw_dri();//实现图像的保存
```

3.7 本章小结

本章节首先对本文所用的嵌入式平台 ZYNQ SoC 进行了介绍，接着实现了基于 ZYNQ SoC 的嵌入式 Linux 操作系统移植。最后设计并实现了基于 ZYNQ Linux 的 OV5640 图像采集输入模块，为系统开发提供了便捷操作，并为构建完整的神经网络应用系统奠定了基础。

第4章 框架设计与实现

4.1 绪论

当前，卷积神经网络技术促使计算机视觉领域中的各类技术得到了巨大的发展，如对象检测、图像识别、语义分割等。许多经典的神经网络模型在图像分类任务中的准确率已经超过了人类（郑冬，2020）。然而，对于嵌入式平台而言，这些模型显得过于庞大，在训练和预测阶段对硬件的性能都有较高的要求，导致了它们难以应用于嵌入式等硬件资源非常有限的平台。为了实现嵌入式神经网络应用的实时性和高效性需求，卷积计算的优化以及轻量化网络架构设计成为了当前的研究热点。本文采用一个完全用 C 和 CUDA 编写的开源神经网络框架 Darknet（Redmon, 2016）作为基础，对其进行扩展和优化，设计出面向 ZYNQ 嵌入式平台神经网络计算框架。为了改善 Darknet 中卷积过程占用硬件平台资源过多、耗时长的问题，本章首先引进了深度可分离卷积算法，然后结合深度可分离卷积设计了更高效的轻量级神经网络模型 MobileNetV2，最后使用 NEON 汇编指令以及 OpenBLAS 对其中的矩阵计算 GEMM 进行了优化。

4.2 基于 DarkNet 框架的卷积计算优化

一个完整的神经网络通常包含多个 CONV 层、Pooling 层和 FC 层，其中 CONV 层为神经网络的核心层，是卷积神经网络中计算强度最大的部分。在 Darknet 框架的卷积层中，采用的是传统的卷积运算，通过“im2col”方法将卷积计算转换为矩阵乘法，然后通过优化矩阵乘法来提高推理性能，这种方式会使得卷积过程产生大量参数，增加了模型的大小和复杂度（于望峰，2020）。

4.2.1 DarkNet 框架简介

Darknet 是一个完全用 C 语言和 CUDA 软硬件并行计算架构编写的开源自定义卷积神经网络框架，最初运行在 GNU/Linux 上，可以在较慢的 CPU 上使用，但可以通过基于 Nvidia 的 GPU 使用 CUDA 工具包进行加速，如果需要的话，还可以将 cuDNN 添加到张量计算工具包中，其计算模式由编译器标志决定。

主要用于图像分类或对视频流中的对象进行实时或接近实时的分类。相对于目前主流的 TensorFlow、Caffe 等框架，Darknet 具备速度快、安装方便等特点，并且，该框架架构简单、无任何依赖项、可移植性好，易于进行改进和扩展，是适合在嵌入式平台部署和开发的理想框架。

4.2.2 DarkNet 中的卷积计算研究

1) im2col_cpu () :

在 Darknet 中，im2col_cpu () 函数的作用是对输入的特征图以及权重进行重排，完成矩阵的向量转换，为其后的 gemm 矩阵乘法作好准备。im2col 是计算机领域中将图片 (image) 转换为矩阵列 (column) 的计算过程，是卷积计算的重要优化方法之一，使用 im2col 方法可以将三维张量转换为二维矩阵，然后利用已实现高效优化的各类 GEMM 库来进行卷积计算加速。但是 im2col 在没有通过精心设计的情况下造成的内存开销较大，其具体实现原理如图 4.1 所示。

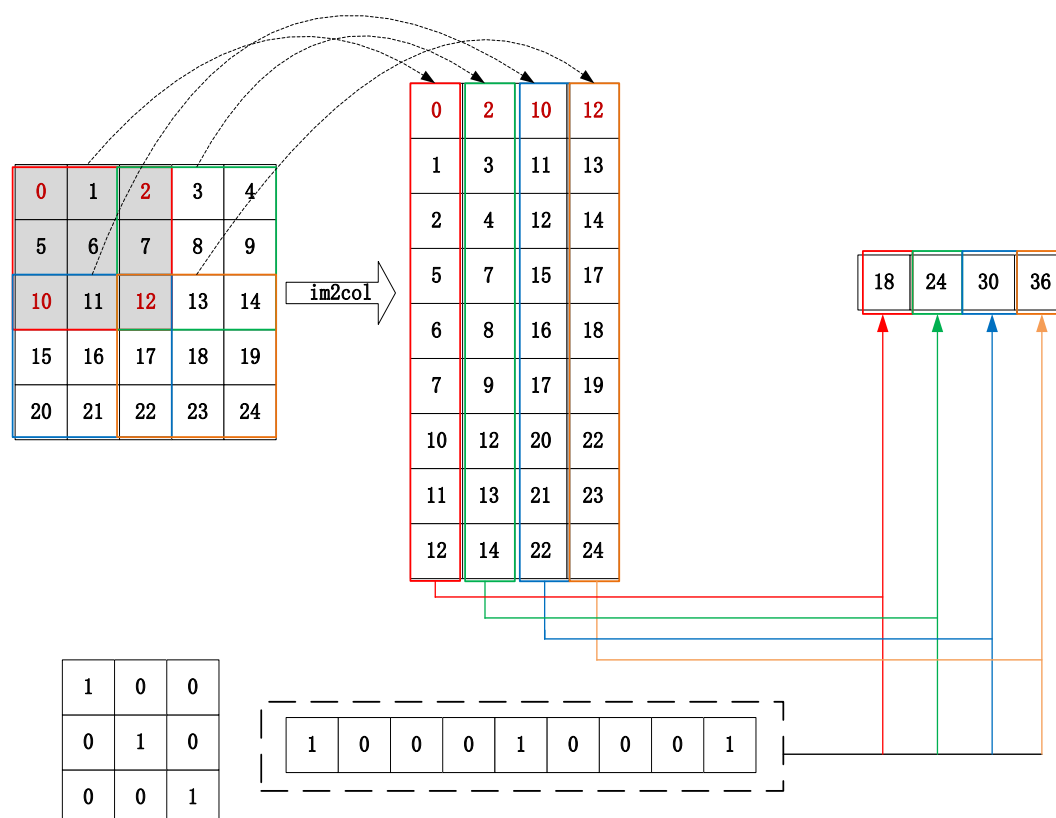


图 4.1 im2col 实现原理

Figure 4.1 Principle of im2col implementation

2) `gemm_cpu()` :

`gemm` 代表了一般的矩阵对矩阵的乘法运算，它本质上是将两个输入矩阵相乘得到一个输出矩阵。不同于我们习惯的矩阵运算，它处理的矩阵通常非常大。例如，一个典型网络中的单层可能需要将一个 256 行、1152 列的矩阵与一个 1152 行、192 列的矩阵相乘，才能得到一个 256 行、192 列的结果。天真地说，这需要 5700 万 ($256 \times 1152 \times 192$) 浮点运算，而在现代架构中可能有几十个这样的层，所以我们经常看到网络需要数十亿次 FLOPs 来计算一个帧。这对于嵌入式平台来说会占用大量的资源，使得大型的神经网络模型无法应用。

4.2.3 深度可分离卷积算法实现

Darknet 同大多数 CNNs 一样，对于对象识别或分类主要进行卷积、池化和全连接三种操作。“`im2col+gemm`”的方式在一定程度上对卷积计算进行了优化，但是会使得卷积过程产生大量参数，不能有效降低模型的大小和复杂度。其主要的缺点是由于构建列矩阵可能会导致空间爆炸，对于一个二维的 $k \times k$ 大小卷积核，产生的列矩阵将比原始输入大 k^2 倍，在资源和算力有限的嵌入式平台上运用具有局限性。近年来，基于神经网络算法的应用逐渐走出实验室转化为实际成果，嵌入式平台得到广泛应用，由于大型神经网络模型在嵌入式平台的适用存在效率问题以及存储问题，研究人员提出了许多轻量神经网络模型，如 SqueezeNet、MobileNet、ShuffleNet、Xception 等等。这些模型都引入了深度可分离卷积的思想，因此本文采用深度可分离卷积替代 Darknet 中的原始卷积，降低网络的复杂性和模型大小，从而提高卷积计算的效率，能够进一步对框架进行扩展，实现更多轻量神经网络模型，使本框架更加适用于移动设备或任何计算能力低的设备。

深度可分离卷积 (depthwise separable convolution) 是许多有效神经网络结构模型的关键结构，其基本思想是将卷积操作分解为两个独立的层，即逐层卷积 (depthwise convolution) 和 1×1 卷积，其中 1×1 卷积也叫逐点卷积 (pointwise convolution)。与传统卷积的卷积核不同，逐层卷积针对每个输入通道采用不用卷积核，即一个卷积核负责一个输入通道，提取每个通道的信息，然后 1×1 逐点卷积将不同通道在相同位置的特征信息进行加权组合，以构建新的特征图

(刘芾 等, 2020)。通过逐层卷积对特征图进行过滤后再通过逐点卷积对信息进行合并的方式, 可有效减少神经网络计算的参数量和模型大小。

如图 4.2 所示, 对于像素为 $D_F * D_F$, 通道为 M 的输入特征图 F , 经过通道数为 M 、大小为 $D_k * D_k$ 的卷积核, 得到 $D_G * D_G * N$ 输出特征图 G 的卷积计算, 假设 $\text{stride} = 1$ 并且使用 padding 的标准卷积, 输出特征图为 (4.1):

$$G_{k,l,n} = \sum_{i,j,m} K_{i,j,m,n} * F_{k+i-1,l+i-1,m} \quad \dots (4.1)$$

其计算成本成倍地取决于输入通道的数量 M 、输出通道的数量 N 、内核大小 $D_k * D_k$ 以及特征图大小 $D_F * D_F$, 代价计算为 (4.2):

$$D_K * D_K * M * N * D_F * D_F \quad \dots (4.2)$$

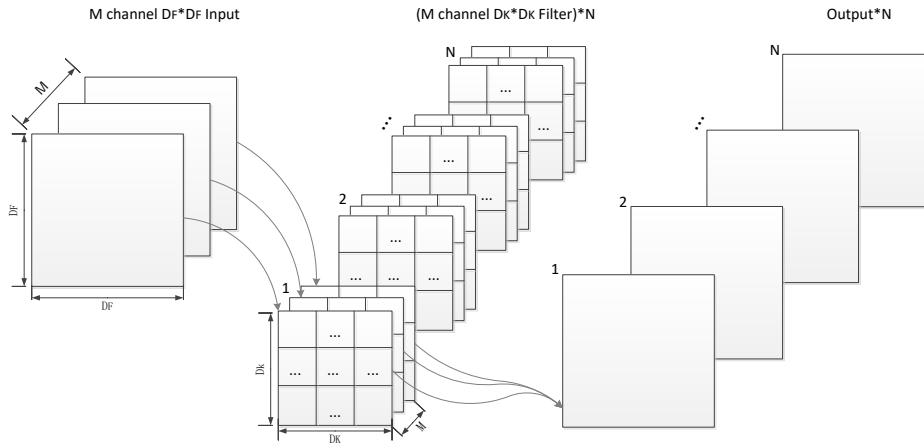


图 4.2 标准卷积

Figure 4.2 Standard CONV

深度可分离卷积的计算过程如图 4.3 所示, 先采用逐层卷积对不同输入通道的信息分别进行过滤, 然后再使用逐点卷积对输出结果进行线性合并。每个输入通道使用一种滤波器的深度卷积可以写成 (4.3):

$$G'_{k,l,m} = K'_{i,j,m} * F_{k+i-1,l+j-1,m} \quad \dots (4.3)$$

其中 K' 是大小为 $D_k * D_k * M$ 的深度卷积核, 将 K' 中的第 m 个滤波器应用于 F 中的第 m 个通道, 得到滤波后的输出特征映射 G' 的第 m 个通道。

逐层卷积的计算代价为式 (4.4):

$$D_K * D_K * M * D_F * D_F \quad \dots (4.4)$$

1*1 卷积的计算代价为 (4.5):

$$M * N * D_F * D_F \quad \dots (4.5)$$

故深度可分离卷积的计算代价为 (4.6):

$$D_K * D_K * M * D_F * D_F + M * N * D_F * D_F \quad \dots (4.6)$$

由此可得, 相同条件下, 深度可分离卷积计算与传统卷积计算的参数量之比为 (4.7):

$$\frac{D_K * D_K * M * D_F * D_F + M * N * D_F * D_F}{D_K * D_K * M * N * D_F * D_F} = \frac{1}{N} + \frac{1}{D_K^2} < 1 \quad \dots (4.7)$$

在神经网络模型中, N 通常远远大于 D_K^2 , 因此由以上计算式子可以得出: 分解为逐层卷积与逐点卷积相结合的深度可分离卷积, 可使计算量显著降低。例如当 $N=128$, $D_K=3$ 时, 深度可分离卷积的计算量仅为传统卷积的 11.89%, 降低了大约 1/8。

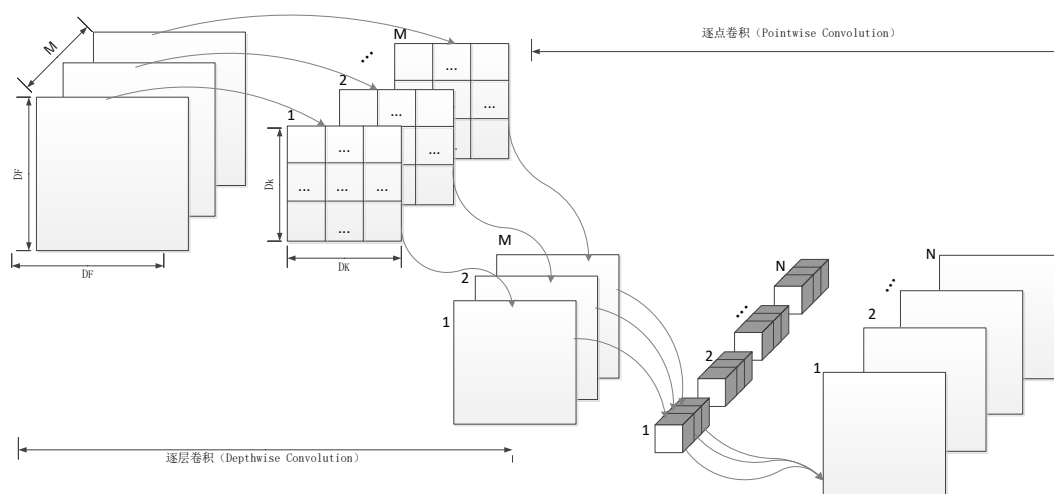


图 4.3 深度可分离卷积

Figure 4.3 Depthwise separable convolution

实现深度可分离卷积的核心代码如下:

```
dw_conv_layer parse_dw_conv (list *options, size_params params)
{
    int size = option_find_int(options, "size", 1); // 卷积核尺寸, 默认是正方形
    int stride = option_find_int(options, "stride", 1); // 卷积步长
    // 是否在输入图像四周补 0, 若需要补 0, 值为 1; 默认设为 0, 不补 0
```

```

int pad = option_find_int_quiet(options, "pad", 0);

int padding = option_find_int_quiet(options, "padding", 0); //四周补 0 的长读
if (pad) padding = size / 2;

// 获取该层使用的激活函数类型,若配置文件中没有指定,则使用 logistic
char *activation_s = option_find_str(options, "activation", "logistic");
ACTIVATION activation = get_activation(activation_s); // get_activation 函数返回枚举类型

int batch, h, w, c; // h, w, c 为上一层的输出的高度/宽度/通道数
h = params.h;
w = params.w;
c = params.c;
batch = params.batch;

if (!(h && w && c)) error("Layer before convolutional layer must output image.");
// 是否进行规范化,1 表示进行规范化,若配置文件中没有指定,则设为 0,即默认不进行规范化

int batch_normalize = option_find_int_quiet(options, "batch_normalize", 0);
//构建深度可分离卷积层
dw_conv_layer layer = make_dw_conv_layer(batch, h, w, c, size, stride, padding,
activation, batch_normalize);

layer.flipped = option_find_int_quiet(options, "flipped", 0);
layer.dot = option_find_float_quiet(options, "dot", 0);
return layer;
}

```

4.3 构建轻量神经网络模型

当前,神经网络技术在各个领域取得了很好的成效,但是,由于不断追求精确度,神经网络计算的模型结构越来越深,复杂度越来越高。然而,在实际的嵌入式应用场景中,庞大的模型难以部署和应用。首先,庞大的模型需要更

多的内存，其次会造成计算速度慢，很难适应于实际嵌入式系统应用。因此，在嵌入式平台设计和部署小型且高效的神经网络模型对于嵌入式神经网络技术落地来说至关重要。本研究在 Darknet 基础上进行扩展，实现了具有可分离卷积、线性瓶颈以及反向残差结构的轻量级高性能神经网络模型 MobileNetV2，以适用于嵌入式设备。

4.3.1 MobileNetV2 网络结构

MobileNetV2 是 Google 发布的神经网络结构，是一个轻量、低延迟，低功耗的模型，快速且易于使用，适用于计算受限的嵌入式设备上。该网络在保证准确性的同时尽可能减少了存储资源和计算能力，成为了用于图像处理的快速网络之一，并且在 CPU 上运行得非常好，而不是昂贵且资源密集的 GPU。MobileNetV2 使用了深度可分离卷积降低参数量和模型大小，并引入了线性瓶颈和反向残差，两者一起构成了瓶颈残差这一基本构建块。MobileNetV2 基本结构如图 4.4 所示。

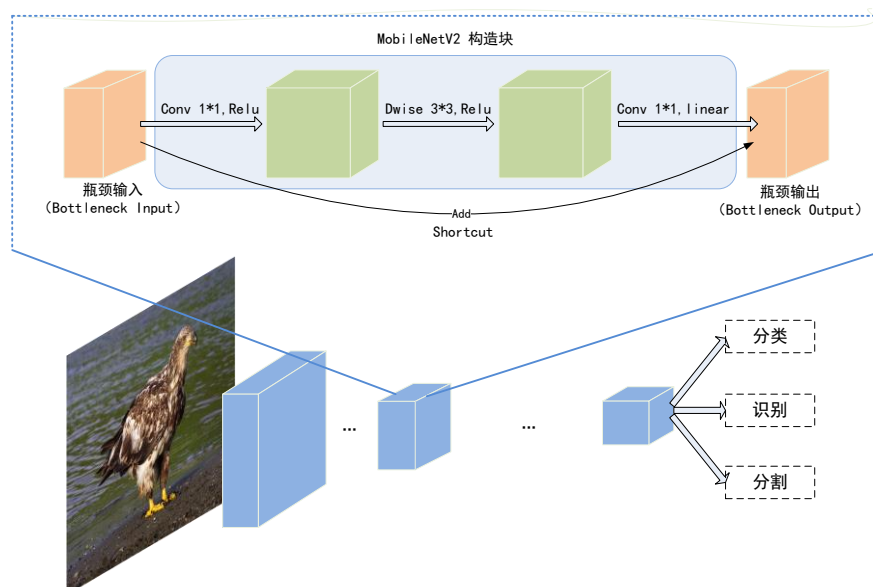


图 4.4 MobileNetV2 结构

Figure 4.4 MobileNetV2 architecture

在 MobileNetV2 中，有两种类型的卷积块。一个是步幅为 1 的剩余块，另一个是步幅为 2 的块以缩小尺寸。两种类型的块都有 3 层。其中，第一层是与 ReLU6 的 1×1 卷积，第二层是在深度方向上卷积，第三层是另一个 1×1 卷积

但没有任何非线性，每一层都具有 BN 操作。据实验表明，如果深层网络再次使用 ReLU，则仅在输出域的非零体积部分具有线性分类器的功能。MobileNetV2 的线性瓶颈结构如图 4.5 所示，其中 t 为通道扩展因子，通常 $t=6$ 。深度卷积在中间。在深度卷积之前是 1×1 卷积，称为扩展层，用来增加通道数量。在深度卷积之后是另一个 1×1 卷积，用来减少通道数，称为投影层或瓶颈层。

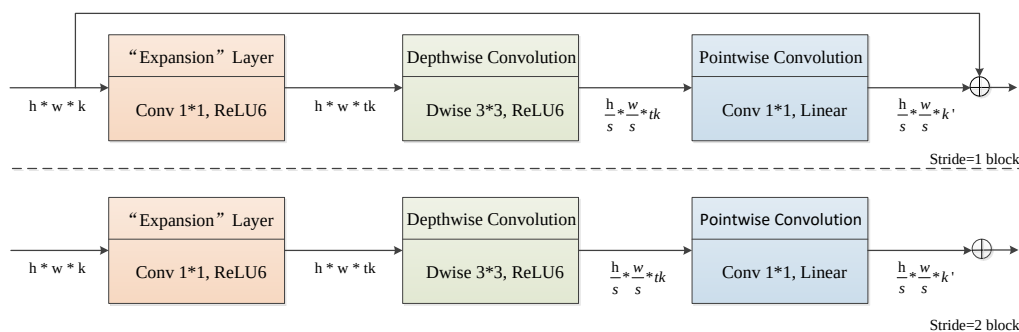


图 4.5 线性瓶颈块

Figure 4.5 linear bottleneck block

4.3.2 反向残差结构

反向残差（Inverted Residual）结构，由 MobileNetV2 CNN 架构提出，有时也称为 MBConv Block，是一种用于图像模型的残差块，为了效率原因使用了倒置结构。

1) 传统残差块

从 AlexNet、VGG、Inception 到残差网络，CNN 的层数开始加深，显示出深层网络比其他技术具有更好的图像分割和分类效果。但是，由于梯度消失和网络退化的现象，训练深层的 CNN 十分困难。使用残差块的深度学习是基于这样一种观点，即近似复杂函数更接近近似。残差网络通过引入残差学习的概念彻底改变了 CNN 架构，并定义了一种用于训练深度网络的有效方法。

（He et.al, 2016）提出残差网络 ResNet，其主要思想是残差块，使用了跳层连接形式实现的残差单元，使信息能够跨层流动，而不会因多次堆叠的非线性变换而导致衰减，从而改善了优化效果，提高了网络性能，解决了增加网络深度带来的网络退化问题。该网络允许开发非常深的神经网络，其中可以包含 100

层或更多层。这是革命性的，因为到现在为止，消失的梯度问题抑制了深度神经网络的发展，而梯度问题在大量层中传播和乘以小梯度时会发生。普通网络和带传统残差单元网络各层的计算方式对比如图 4.6 所示。

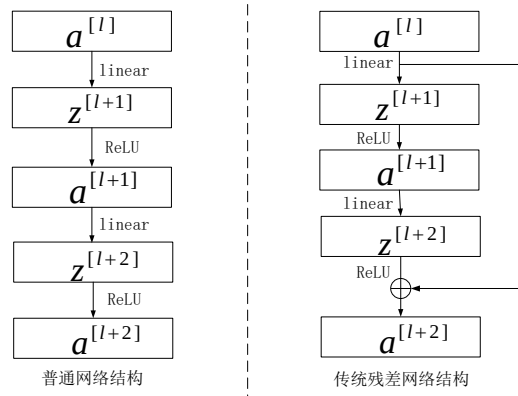


图 4.6 网络层计算方法

Figure 4.6 Network layer computing method

残差网络通过额外的连接，使得梯度可以更容易向后传播。残差单元是一个灵活的模块，可以扩展网络的能力，在不增加训练复杂度的情况下，使网络变得更深。将以前的 ImageNet 获奖者的误差值与 ResNet 的误差值进行比较，我们可以看到性能有了明显的提高。AlexNet（2012）的 top-5 错误为 15.3%，其次是 ZFNet（2013）的 top-5 错误为 14.8%（功能可视化），再次是 GoogleNet（2014）的 top-5 错误，误差为 7.8%，然后是 ResNet（2015），其精度首次低于 5%。描述网络计算的方程式如表 4.1 所示

表 4.1 网络层计算表达式

Table 4.1 Network layer computing expression

普通网络	传统残差网络
$a^{[l]} = g(z^{[l]})$	$a^{[l]} = g(z^{[l]})$
$z^{[l+1]} = W^{[l+1]}a^{[l]} + b^{[l+1]}$	$z^{[l+1]} = W^{[l+1]}a^{[l]} + b^{[l+1]}$
$a^{[l+1]} = g(z^{[l+1]})$	$a^{[l+1]} = g(z^{[l+1]})$
$z^{[l+2]} = W^{[l+2]}a^{[l+1]} + b^{[l+2]}$	$z^{[l+2]} = W^{[l+2]}a^{[l+1]} + b^{[l+2]}$

$$a^{[l+2]} = g(z^{[l+2]})$$

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

2) 反向残差结构

传统残差块为宽->窄->宽的结构以及通道数，即先将输入特征图进行降维，然后提取特征，最后再升维。相反，反向残差块采用窄->宽->窄结构，因此是倒置的，故称为反向残差结构。该结构先是用 $1*1$ 卷积对输入特征图进行升维，然后利用深度可分离卷积提取输入特征，最后再用 $1*1$ 卷积降低到原始维度，在 $\text{Stride}=1$ 的 block 中加入了跳跃连接，使得网络在较深的时候仍然可以进行训练。为了弥补深度可分离卷积中逐层卷积在低维空间提取特征效果较差的缺点，在逐层卷积前多加入了一个 1×1 的逐点卷积，来完成对卷积神经网络中的特征图进行升维的任务，从而丰富特征数量，进一步提高精度，于是形成了逐点卷积-逐层卷积-逐点卷积的结构。传统残差结构与反向残差结构的对比如图 4.7 所示。

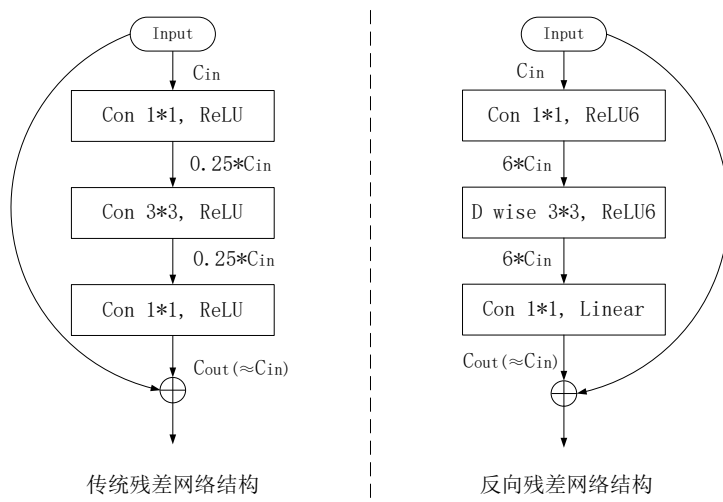


图 4.7 网络结构对比

Figure 4.7 Network architecture comparison

4.3.3 线性瓶颈结构

瓶颈层是指相关信息在卷积层中的低维表示，可用于预测特征和分类。由于许多连续的矩阵乘法往往不能简化为一个数值运算，因此在神经网络中通常

使用非线性激活函数，在连续的卷积层之间折叠通道信息，从而允许构建深度的网络。常用的非线性激活函数是线性整流函数（Rectified Linear Unit, ReLU），将 ReLU 应用到神经网络内连续的卷积层上，会造成这些层之间固有的信息丢失。对于任意 n 维的卷积网络层 L_i ，形成对应于相关信息的抽象的一组激活张量 T ；人们认为，对于任何一个 L_i ，都存在完全嵌入 T 的低维流形 $M \in \mathbb{R}^n$ 。如果 M 存在于高维激活空间中，则产生非零 M 的 ReLU 操作与关于 ReLU 的输入的线性变换一致，因此 ReLU 操作可以保存与输入或 M 相关的信息。因此，假设 M 是低维的并且存在于更高维的激活空间中，通过简单地不对其应用非线性激活函数，可以将瓶颈视为线性操作。

在 MobileNetV2 网络的残差单元中，最后一层使用了 1×1 卷积，降低了通道数量，造成了特征被压缩。为了保证模型的表达能力，解决 ReLU 非线性激活函数使得低维度信息丢失，破坏有用信息的问题，使用了线性瓶颈（Linear Bottlenecks）结构，即在低维度特征图中用线性（Linear）激活函数来替代 ReLU，因此被称为线性瓶颈（Linear Bottleneck）结构。已有实验表明，使用线性瓶颈可提高性能，尤其是在分类任务中。

4.3.4 MobileNetV2 的实现

与 Caffe 框架类似，darknet 框架通过一个配置文件就可以定义一个模型的架构。DarkNet 的网络配置文件为 .cfg 文件，网络中不同类型的层通过 [] 内参数进行识别，例如 [convolution] 代表卷积层、[upsample] 代表下采样层，其后紧跟本层采用的激活函数、卷积核、stride 及 padding 的大小参数等。其中 [net] 层为配置网络超参数的特殊层，该层包括学习率、网络的通道数、迭代次数等一系列参数的配置。本文根据 MobileNetV2 的网络结构（如）在面向 ZYNQ SoC 平台的神经网络计算框架中通过添加自定义 mobilenetv2.cfg 文件，设置网络结构及各层参数来构建 MobileNetV2 神经网络模型。由于 DarkNet 框架中没有 ReLU6 激活函数的实现，因此要构建 MobileNetV2 模型还需要在框架中实现 ReLU6 激活函数的计算。ReLU6 的计算方式如式（4.8）所示，函数图像如图 4.8 所示。

$$\text{ReLU6} = \min(6, \max(0, x)) \quad \dots (4.8)$$

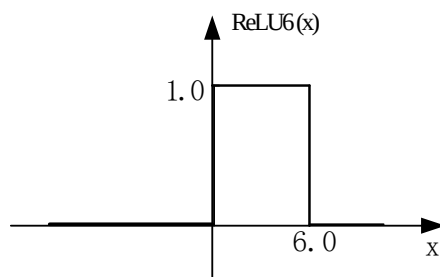


图 4.8 ReLU6 激活函数

Figure 4.8 ReLU activation function

实现该函数的主要代码为：

```
static inline float relu6_activate(float x) { //通过 inline 快速实现计算
    return min_val_cmp(max_val_cmp(x, 0), 6); }
static inline float relu6_gradient(float x) { return (x > 0 && x < 6); }
```

MobileNetV2 网络结构如表 4.2 所示。

表 4.2 MobileNetV2 网络结构

Table 4.2 MobileNetV2 architecture

Input	Operate	t	c	n	s
224*224*3	conv2d	-	32	1	2
112*112*32	bottleneck	1	16	1	1
112*112*16	bottleneck	6	24	2	2
56*56*24	bottleneck	6	32	3	2
28*28*32	bottleneck	6	64	4	2
14*14*64	bottleneck	6	96	3	1
14*14*96	bottleneck	6	160	3	2
7*7*160	bottleneck	6	320	1	1
7*7*320	conv2d	-	1280	1	1
7*7*1280	avgpool	-	-	1	-
1*1*1280	conv2d	-	k	-	-

*注：t：扩张倍数；c:输出通道数；n:重复次数；s:步长

MobileNetV2 模型在框架中的部分参数配置文件如图 4.9 所示。

```

192.168.93.14 - PuTTY
[net]
batch=1
subdivisions=1
height=224
width=224
channels=3
momentum=0.9
decay=0.0005
max_crop=256

learning_rate=0.01
policy=poly
power=4
max_batches=600000

angle=7
hue=.1
saturation=.75
exposure=.75
aspect=.75

192.168.93.14 - PuTTY
# conv1
[convolutional]
filters=32
size=3
pad=1
stride=2
batch_normalize=1
activation=relu6

# conv2_1_expand
[convolutional]
filters=32
size=1
stride=1
pad=0
batch_normalize=1
activation=relu6

192.168.93.14 - PuTTY
# conv2_1_dwse
[convolutional]
groups=32
filters=32
size=3
stride=1
pad=1
batch_normalize=1
activation=relu6

# conv2_1_linear
[convolutional]
filters=16
size=1
stride=1
pad=0
batch_normalize=1
activation=linear

```

图 4.9 训练参数配置

Figure 4.9 Training parameters configuration

4.4 GEMM 计算的 NEON 优化

神经网络或卷积神经网络的绝大多数计算时间花费在执行 GEMM（通用矩阵乘法）算法上(Jia, 2014b)。本文通过 perf 软件对 DarkNet 运行时的函数进行分析，如图 4.10 所示，GEMM 计算占用系统资源最多，且为计算耗时最长的一个函数。GEMM 在计算中总是使用重复的数据，例如，每行与每列相乘。任何 GEMM 算法都会多次访问每个元素。充分利用该属性可减少数据移动和存储，围绕该特征设计神经网络计算架构将产生更大的效益。

```

Samples: 155K of event 'cycles:ppp', Event count (approx.): 1850924551
Overhead Symbol
70.77% [.] gemm_nn_omp_fn.0
1.34% [.] im2col_cpu
1.29% [.] im2col_get_pixel
0.92% [.] activate
0.65% [.] rand_normal
0.50% [.] copy_cpu
0.16% [.] activate_array
0.12% [.] fill_cpu
0.11% [.] make_convolutional_layer
0.09% [.] add_bias
0.09% [.] shortcut_cpu
0.08% [.] normalize_cpu
0.02% [.] scale_bias
0.02% [.] rand@plt
0.01% [.] upsample_cpu

```

图 4.10 性能分析

Figure 4.10 Performance analysis

4.4.1 ARM Cortex-a9 微处理器

ZYNQ 的处理器系统包含了一个双核 ARM Cortex-A9 处理器，还有一组相关的处理资源，共同形成应用处理器单元（Application Processing Unit，APU）。APU 中的每个 ARM 处理核分别关联了一个 NEON 与浮点扩展单元、内存管理单元及一个一级 cache 存储器。Cortex-A9 处理器是 ARMv7 体系结构，支持称为 NEON 的高级单指令多数据（Single Instruction Multiple Data，SIMD）扩展，可以处理存储在 64 位双字寄存器和 128 位四字寄存器中的数据，加速多媒体、信号处理应用、图形和图像处理算法。NEON 指令是对标准 ARM 指令集的扩展，可以直接使用或者通过写出特定格式的 C 代码，来让编译器产生 NEON 指令。SIMD 术语意味着 NEON 引擎可以对输入向量中的多组数据，同时执行相同的运算来得到对应的输出向量。这种计算范式很好地迎合了像图像和视频处理这样的应用，可以同时大量的数据样本（像素点）做运算。

4.4.2 基于 NEON 汇编指令的 GEMM 实现

Cortex-A9 处理器是性能和功耗优化的多核处理器，并且是 Arm 部署最广泛且最成熟的应用程序处理器之一。使用 ARM NEON 优化算法可以利用现成的开源库，或者使用编译器的自动向量化以及 NEON 内联和 NEON 汇编四种方式。前三种方式虽然实现简单，但是受限于编译器的优化能力，性能往往很难达到极致。为了使 NEON 能够进行高性能计算，本文主要使用 NEON 汇编技术来对卷积计算当中的矩阵乘法进行优化。

SIMD 是一种指令集体系结构，具有一个控制单元和一个以上处理单元，它们通过在处理单元上执行单个指令流，控制生成所有对处理单元的控制信号，并通过它们对不同的数据流执行相同的操作，实现数据级并行性。反之，使用一条指令处理每个单独数据的常规方法称为 SISD（Single Instruction Single Data）。SISD 和 SIMD 操作之间的差异如图 4.11 所示。

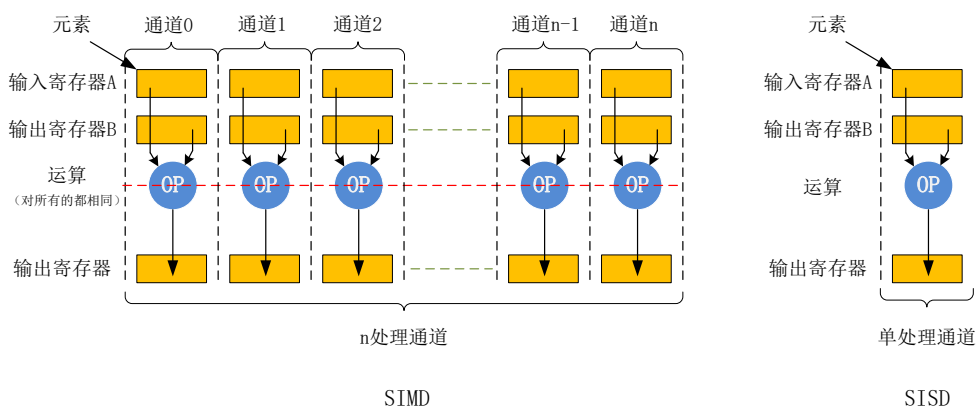


图 4.11 SISD 与 SIMD 对比

Figure 4.11 Comparison of SISD and SIMD

在 Cortex-A 系列处理器上，如果不采用 SIMD 转换，大量时间将花费在处理 8bit 或 16bit 的数据上，但是处理器本身的加法逻辑单元，寄存器，数据深度主要都是为了 32 位的运算而设计的，因此，NEON 应运而生。NEON 就是一种基于 SIMD 思想的 ARM 技术，在 ARMv7 架构中开始被采用，目前可以在 ARM Cortex-A 以及 Cortex-R 系列处理器中使用。

与高级语言类似，ARM 支持对不同数据类型的操作。我们可以加载（或存储）的数据类型可以是有符号的和无符号的整数、未指定类型的数、浮点数和在[0,1]上的多项式。NEON 的数据类型如表 4.3 所示。

表 4.3 NEON 数据类型

Table 4.3 Data type of NEON

类型	8-bit	16-bit	32-bit	64-bit
无符号整形	U8	U16	U32	U64
有符号整形	S8	S16	S32	S64
未指定类型整数	I8	I16	I32	I64
浮点数	不可用	不可用	F(或 f32)	不可用
[0,1] 上的多项式	P8	P16	不可用	不可用

ARMv7 架构下为 ARM32 位寄存器，其中包含了 15 个 ARM 通用寄存器 r0~r14、一个程序计数器 PC 以及多个 NEON 寄存器。NEON 指令主要是通过操

作 NEON 寄存器来实现并行加速的。NEON 允许 64-bit 或 128-bit 并行，它的寄存器库可以看作 32 个 64-bit 长的寄存器 D0-D31，每个寄存器 D 可以存储两个浮点型数据，或者也可以看作为 16 个 128 位长的寄存器 Q1-Q15，每个寄存器 Q 可以存储 4 个浮点型数据。这两种寄存器是重叠的，每个 Q1-Q15 寄存器都映射到一对 D 寄存器，其对应关系为 $Q_n = D_{2n} + D_{2n+1}$ 。可以看出，一个指令最多可同时进行 4 个乘法计算，理论上可以将速度提升 4 倍。相对于单指令单数据的计算方式，NEON 寄存器通过一次实现多个数据处理从而节省数据访存的时间，提高了计算效率。NEON 寄存器的示意图如图 4.12 所示。

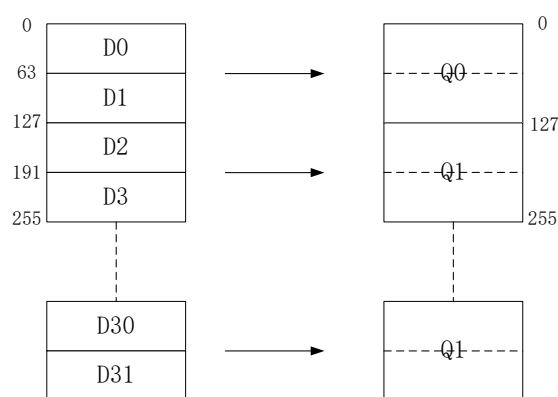


图 4.12 NEON 寄存器

Figure 4.12 NEON register

ARM 指令通常遵循一个或两个操作，通常使用以下格式：

MNEMONIC{S}{<condition>} <Rd>, <Rn> {, <Operand2>}

ARM 指令集具备灵活性，并非所有指令都使用模板中提供的所有字段，其中 {} 是可选项。模板中每个字段的含义如下：

MNEMONIC：指令的缩写（指令助记符）

{S}：一个可选的后缀。表示是否影响 CPSR 寄存器的值

{<condition>}：更新条件标志-执行指令需要满足的条件

<Rd>：存储指令结果的寄存器（目标）

<Rn>：第一个操作数的寄存器

{<Operand2>}：第二（灵活）操作数。可以是立即数（数字）或带可选移位的寄存器。

GEMM 运算中存在大量乘加操作，这在硬件实现起来比较复杂。通过使用 perf 对 GEMM 的计算进行分析，得出该计算中的数据加载以及乘法操作占用的时间和资源最多，因此本部分利用 NEON 的 SIMD 操作对 GEMM 计算进行优化，获得了读数和乘法的效率提升。实现优化的关键是将特征图与卷积核的卷积计算循环使用矢量加载指令 vld1.32 以及矢量乘加指令 vmla.f32 进行并行加载和乘加。vld1.32 可以从内存地址中同时读取 4 个 32-bit 的数到 NEON 的 Q 寄存器，vmla.f32 可以同时进行四个数的乘加操作。以 (4.9) 的矩阵乘法为例：

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{21} \\ b_{31} \\ b_{41} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} + a_{14}b_{41} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} + a_{24}b_{41} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} + a_{34}b_{41} \\ a_{41}b_{11} + a_{42}b_{21} + a_{43}b_{31} + a_{44}b_{41} \end{bmatrix} \dots (4.9)$$

其实现原理如图 4.13 所示，一次循环通过 vld1.32 {q1}, [r1]! 指令每次从内存中读取四个数加载到 Q1 寄存器，操作完成后 r1 进行更新继续读取第二列的四个数；然后再使用 vmla.32 q2,q1,d0[0]实现 q1 中的四个数与 d0[0]同时进行乘累加，结果存储在 Q2 寄存器，仅需要四次循环就可以计算出两个矩阵相乘的结果。

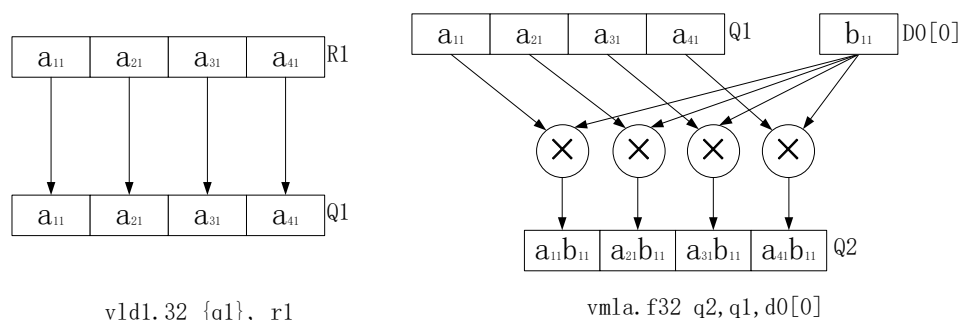


图 4.13 NEON 加速原理

Figure 4.13 Acceleration principle of NEON

4.5 GEMM 计算的 OpenBLAS 加速实现

OpenBLAS 是基本线性代数子程序 (Basic Linear Algebra Subprograms, BLAS) 的优化实现。BLAS 的功能可划分为三个层次：level 1 向量运算 (axpy) $y \leftarrow \alpha x + y$ 、level 2 矩阵-向量运算 (gemv) $y \leftarrow \alpha Ax + y$ 和 level 3 级矩阵运算

(**gemm**) $y \leftarrow \alpha AB + \beta C$ 。这三个层次分别对应着不同的算法复杂度，层次越高复杂度越高。BLAS 执行许多有用的线性代数任务，包括向量范数，矩阵向量和矩阵矩阵乘法。

GEMM 执行的矩阵乘累加操作为 $C = \beta C + \alpha AB$ ，其中 A、B、C 分别是大小为 $M \times K$ 、 $K \times N$ 、 $M \times N$ 的矩阵， α 、 β 为标量。与之相对应的伪代码为：

```
for ( int m = 0; m < M; m++)
for ( int n = 0; n < N; n++){
    C[m][n] = 0;
    for ( int k = 0; k < K; k++){
        C[m][n] += A[m][k] * B[k][n];
    }
}
```

虽然这种操作在算法上很简单，一个三层深度的循环嵌套就足以完成，但由于现代处理器上存在多级内存层次结构，高性能实现可能相当复杂。

Openblas 进行加速的基本方法有以下三种：

1) 利用 **blocking**：将大矩阵相乘切分成小矩阵相乘，充分利用 **cache**，提升数据局部性，使得所有运算能在 **cache** 里完成；

2) 利用 **packing**：将矩阵存储在连续空间中，提高访存性能，可使得 **cache** 的效率非常高；

3) 优化小矩阵相乘：以 4×4 的矩阵为基本单位，使用寄存器存储运算中的变量，而不是操作内存，从而减轻 **cache** 的压力，利用指针移动代替矩阵下标的计算，然后利用向量化指令进行最小矩阵的乘加运算。

OPENBLAS 库可以由 C，C++ 或 Fortran 程序调用。对于 C 和 C++ 调用程序，有两种选择：

1) F2C 接口：要求用户将每个 OPENBLAS 函数声明为一个外部函数，在函数名称后添加下划线，并通过引用调用每个参数，如下所示：

```
dgemm_ (&transa_char, &transb_char, &m, &n, &k, &alpha, a,
        &lda, b, &ldb, &beta, c, &ldc);
```

2) C 接口：要求用户在字符串 **cblas_** 之前添加每个函数的名称，并插入一个附加的初始参数，该参数指定矩阵存储是行优先还是列优先，并允许按值传

递标量输入参数:

```
cblas_dgemm (CblasColMajor, transa, transb, m, n, k, alpha, a, lda,
b, ldb, beta, c, ldc) ;
```

在 DarkNet 框架中, GEMM 矩阵运算由 `gemm.c` 实现, 利用 OpenBlas 只需对 `src` 目录下的 `gemm.c` 代码进行修改, 以 C 接口的方式进行调用即可。

第一步: 在代码文件开头添加如下代码:

```
#if OPENBLAS
#include "cblas.h"
#endif
```

第二步: 修改 `gemm` 函数, 添加 OpenBLAS 接口, 代码如下所示:

```
//若开启 OPENBLAS 优化
#if OPENBLAS
//数组在内存中的存储方式为行优先
//M: 表示 A 或 C 的行数。N: 表示 B 或 C 的列数。K: 表示 A 的列数或
B 的行数 (A 的列数=B 的行数)
// ALPHA 一个标量, 通常为 1; BETA: 一个标量参数, 通常设置为 0
if(!TA && !TB) //矩阵 A 和 B 都不进行转置
    cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans, M, N, K,
ALPHA, A, lda, B, ldb, BETA, C, ldc);
else if (TA && !TB) //矩阵 A 转置, B 不进行转置
    cblas_sgemm(CblasRowMajor, CblasTrans, CblasNoTrans, M, N, K,
ALPHA, A, lda, B, ldb, BETA, C, ldc);
else if (!TA && TB) //矩阵 A 不进行转置, B 进行转置
    cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasTrans, M, N, K,
ALPHA, A, lda, B, ldb, BETA, C, ldc);
else //矩阵 A 和 B 都进行转置
    cblas_sgemm(CblasRowMajor, CblasTrans, CblasTrans, M, N, K,
ALPHA, A, lda, B, ldb, BETA, C, ldc);
```

```
#else  
    gemm_cpu( TA,  TB,  M, N, K, ALPHA,A,lda, B, ldb,BETA,C,ldc);  
#endif  
}
```

4.6 本章小结

本章节基于开源神经网络框架 **DarkNet** 进行了改进，引入了深度可分离卷积算法并在其基础上实现了轻量神经网络模型 **MobileNetV2**。此外，还利用了 ZYNQ SoC 具备的 ARM Cortex-a9 双核的优势，使用 NEON 汇编指令以及 OpenBLAS 对框架中耗时最长、占用资源最多的 GEMM 算法进行了优化。

第5章 测试与分析

5.1 实验设计

前面分别对图像采集模块、卷积计算优化、矩阵乘法的优化实现进行了介绍。本章将介绍利用前面的设计实现的神经网络计算框架部署到 ZYNQ-7020 SoC 嵌入式平台进行图像分类测试。具体实验流程如下：

- 1) 开发环境部署。
- 2) 数据集准备。
- 3) 下载源码并进行卷积计算优化，构建基于深度可分离卷积的轻量神经网络模型 MobileNetV2，使用 NEON 汇编指令以及 OpenBLAS 对 GEMM 计算进行优化。
- 4) 进行权重训练和开发板测试，比较分析实验结果。

5.1.1 开发环境部署

1) 硬件环境

本文采用了 Xilinx 的 ZYNQ-7020 SoC 作为目标平台，并搭载了一个 OV5640 摄像头。ZYNQ-7020 SoC 是以 ARM Cortex-a9 双核处理器为核心的一个开发板形态产品。ARM Cortex-a9 处理器采用 armv7 架构，可搭载 Linux 操作系统，并且具有内存，适合作为本实验的目标平台。其核心参数为：

- ① ZYNQ-7000 XC7Z020-1CLG400C，主要由 ARM Cortex-A9 双核和 7 系列 FPGA 构成；
- ② 存储介质：512MB 的 DDR3 内存颗粒；
- ③ 接口：JTAG、USB_UART PHY、千兆以太网 PHY、HDMI PHY、OV5640 高清摄像头、高速 ADC/DAC 等；
- ④ 电源：5v（可选电压 1v、1.3v、1.8v、3.3v）；
- ⑤ 板上时钟：50MHz LVCMOS 晶振（PL 端）、33.333MHz LVCMOS 晶振（PS 端）。

在实验过程中通过串口或 SSH 远程登录控制开发板的操作系统，使用本文

设计实现的神经网络计算框架进行图像分类实验。由于嵌入式平台资源有限，实现神经网络训练十分困难，故本实验在开发板的基础上，还需要一台主机，主要进行神经网络的训练，以及优化的代码编写调试和程序编译，最终将代码以及训练好的权重数据下载到 ZYNQ 平台。主机处理器为 Intel(R) Core(TM) i5-8265U，内存为 8G DDR3 1600MHZ，同样装有 Linux 操作系统（Ubuntu18.04.01）。

2) 软件环境

① 开发工具：采用 Xilinx 官方提供的开发工具套件，包含 2018.02 版本的 Vivado HLs 以及 SDK。

② 交叉编译工具 arm-linux-gnueabi-hf-gcc：利用主机编译目标平台程序

③ DarkNet 开源代码：以该框架为基础进行扩展和优化

④ perf：分析代码，找到消耗资源和运行时间较长的程序进行优化

⑤ OpenBLAS 函数库：编译完成后由程序调用

5.1.2 图像分类测试集

ImageNet (Jia et al., 2009) 是一个超过 1500 万张标签高分辨率图像用于基准图像分类算法的最广泛使用的大规模数据集之一，数据集中的图像被整理为一个层次结构，该层次结构中的每个节点由成百上千个图像进行描述。ImageNet 数据集包括训练数据、验证数据和图片标签三个部分，验证数据和测试数据不包含在 ImageNet 训练数据中（重复项已移除）。从 2010 年开始，作为 Pascal 视觉对象挑战赛的一部分，ImageNet 举办了一年一度的大型视觉识别挑战赛 ILSVRC。ILSVRC 使用 ImageNet 的一个子集，在 1000 个类别中的每个类别中大约有 1000 张图像。总共大约有 120 万张训练图像，5 万张验证图像和 15 万张测试图像。ILSVRC-2012 通常用于图像分类，在本文的实验当中选择该数据集进行训练和测试。下载完成后通过以下步骤制作为 DarkNet 框架适用格式：

1) 获得数据集后首先进行解压：

```
tar -xzf ILSVRC2012_bbox_val_v3.tgz
```

```
mkdir -p imgs && tar xf ILSVRC2012_img_val.tar -C imgs
```

2) 使用 `imagenet_label.sh` 脚本文件借助图像注释对图像进行标记:

```
bash imagenet_label.sh
```

3) 以上步骤将生成两个内容: 一个文件包含与图像的重命名符号链接, 另一个为包含标记图像路径列表的文件。然后将 `labelled/inet.val.listdata/` 文件移动到 Darknet 中的子目录:

```
mv inet.val.list <path-to>/darknet/data
```

5.1.3 评价指标

1) Top-1 和 Top-5: Top-1 和 Top-5 精度在神经网络的图像分类任务中常被作为图片检测准确率的评价指标, 如 (Redmon, 2016, Redmon and Farhadi, 2018, Russakovsky et al., 2015)。Top-1 精度是常规精度, 这意味着模型答案 (概率最高的答案) 必须恰好是预期答案。Top-5 准确性意味着模型给出的 5 个最高概率答案之一要与预期答案相匹配。假设正在使用机器学习算法来使用神经网络进行对象识别。显示了一张猫的图片, 神经网络的输出为: 老虎 0.4、狗 0.3、猫 0.1、狮子 0.08、鸟 0.02, 若使用 Top-1 精度, 此输出将被视为错误, 因为被识别成了老虎; 若使用 Top-5 精度, 则此输出将被视为正确, 因为猫排在前五位的预测当中。

2) Ops (operations per second): 常在使用到大量浮点运算的科学领域用来估算电脑执行效能, 在 DarkNet 中表示为 BFLOPs, 其含义是 Billion Float Operations (Redmon and Farhadi, 2018), 用来描述某次卷积运算需要的多少个十亿次浮点运算, 通过将卷积运算等计算耗费的 BFLOPS 加起来就可以表示模型的复杂度。计算公式为: $l.bflops = (2.0 * l.n * l.size * l.size * l.c * l.out_h * l.out_w) / 1000000000$; `l.n`、`l.size`、`l.c`、`l.out_h`、`l.out_w` 分别表示卷积核的数目、大小、个数以及卷积后的图像高度和宽度。

3) Paramas: 表示训练权重的大小, 通过此参数可以判断模型的权重占用内存资源情况。

5.2 板上测试及分析

将实现深度可分离卷积算法、轻量神经网络模型 MobileNetV2、使用 NEON

汇编以及 OpenBlas 进行矩阵乘法运算 GEMM 优化后的神经网络计算框架移植到开发板中，下载 PC 端训练好的权重至开发板<path>/darknet 目录下并修改 Makefile 文件，进行编译。

5.2.1 功能性测试

本文使用第四章实现的 MobileNetV2 网络模型作为特征提取骨干网络，通过 PC 端进行 ImageNet 数据集的权重训练，然后下载到开发板上。使用第三章设计并实现的 OV5640 图像采集输入单元进行图像采集，然后对单幅图像在开发板上进行图像分类实验，验证该网络的准确性以及系统的完整性。测试结果如表 5.1 所示。

表 5.1 功能性测试结果

Table 5.1 Functionality testing

测试图像	测试结果	
	Pomeranian	56.18%
	Samoyed	10.16%
	Chihuahua	10.16%
	Arctic fox	4.08%
	Pomeranian	1.99%
	Minibus	57.51%
	Recreational vehicle	21.41%
	Moving van	8.86%
	Garbage truck	4.75%
	Bus	4.27%

5.2.2 性能测试

本文在 ImageNet 图像分类数据集上对基于本研究中实现的计算框架构建的 MobileNetV2 网络模型与 AlexNet 和 Darknet-19 进行实验对比。在实验过程中，只开启 ARM CPU 进行计算，除网络结构不同以外，其他条件均保持一致。在实验中，对不同网络的 Top-1、Top-5、单张图片识别时间、计算复杂度、参数量大小 5 个指标进行对比，结果如表 5.2 所示。可以看出，使用该模型进行图像分类识别，相对于 AlexNet 模型，MobileNetV2 识别单张图片时间减少 50%、浮点运算次数减少 60%、参数量减少 94%，同时保持了 90.3%的 Top-5 精度；相对于 darknet-19 模型， MobileNetV2 识别单张图片时间减少 84% 、浮点运算次数减少 88% 、参数量减少 83% ，同时保持了 90.3%的 Top-5 精度。具有一定优势。

表 5.2 性能测试

Table 5.2 Performance test

数据集	网络	Top-1	Top-5	Times	Ops	Paramas
ImageNet	AlexNet	72.9	91.2	25.646	2.153Bn	249.5MB
	Darknet-19	70.7	89.9	83.936	7.291Bn	83.4MB
	MobileNetV2	71.4	90.3	12.731s	0.858Bn	14.2MB

5.2.3 执行时间测试

本节分别使用 AlexNet、Darknet-19 和 ResNet18 框架对同一幅图像进行分类，分别测试使用 ARM CPU、ARM NEON 汇编指令优化和 OpenBLAS 优化三种方案的前向计算耗时。MobileNetV2 由于使用了深度可分离卷积，故不参与优化方案的速度评估，除网络结构不同，其他条件均保持一致，测试的结果如表 5.3 所示，可以看出，只使用 ARM CPU 进行计算耗时最高，相对于 ARM CPU，使用 ARM NEON 汇编指令的朴素优化将计算速度提升了约 40%，而 OpenBLAS 库的优化速度提升了 80%左右。

表 5.3 执行时间测试

Table 5.3 Execution time test

模型\耗时 (s)	ARM CPU	NEON	OpenBLAS
AlexNet	25.646	16.360	4.490
Darknet-19	83.936	50.601	13.179
ResNet18	54.464	33.058	9.788

5.3 本章小结

本章对前两章实现的 OV5640 图像采集输入模块与基于 DarkNet 框架实现的轻量神经网络模型 MobileNetV2、GEMM 矩阵乘法运算的 ARM NEON 汇编指令及 OpenBLAS 加速两种优化方案进行了准确度测试、性能评估以及优化方案的耗时对比。通过实验可以看出，本文基于 DarkNet 设计实现的神经网络计算框架在资源消耗、计算速率方面有较大提升，并且保持了一定的精确度，适合在嵌入式端进行部署。

第6章 总结与展望

6.1 工作总结

当前, 神经网络技术极大地提高了语言、视觉和许多其他领域的技术水平, 被广泛应用于许多领域。现有的神经网络计算框架执行推理需要很多资源, 如设备执行捕获数据预处理然后给神经网络以执行大量的浮点操作。部署这样一个系统在云端, 由于存在到服务器的往返过程会产生更多的延迟, 数据需要离开设备时存在隐私风险, 数据需要发送到服务器时存在网络连接会产生电力消耗。这些问题都可以通过在微控制器、嵌入式移动设备等边缘设备上部署神经网络计算框架来解决。因此如何将神经网络技术应用于嵌入式系统成为当前研究人员重点关注的一个领域。针对现有框架所需计算资源多、计算量大等问题, 本文基于 DarkNet 开源框架进行扩展和优化, 设计并实现了面向 ZYNQ 嵌入式移动平台的神经网络计算框架, 实验证明该框架在保持精度的前提下有效提高了神经网络的计算效率, 并且系统资源占用较低。

为了构建完整的神经网络应用系统, 本文设计并实现了图像输入采集模块。针对嵌入式平台裸机开发难度大的问题, 采用面向操作系统开发的方式, 实现了 ZYNQ SoC 的嵌入式 Linux 操作系统移植, 降低开发难度。图像采集单元的实现可以从实际场景中进行图像采集通过 VDMA 传输到 SD 卡中以供神经网络计算框架使用。通过测试, 可以看出采集的图片通过神经网络计算进行识别取得了较高的识别精度。

针对现有计算框架算力需求大、参数量多, 难以在嵌入式平台进行部署的问题, 本文在 DarkNet 基础上引入了深度可分离卷积算法, 并实现了利用深度可分离卷积、线性瓶颈以及反向残差结构构建的 MoblieNetV2 神经网络模型, 降低模型复杂度, 以提高卷积网络计算效率。此外, 针对卷积计算的核心算法 GEMM 占据的大量的运算时间和资源的问题, 本文采用了 ARM NEON 汇编指令以及 OpenBLAS 对其进行了优化。通过图像分类测试可以证明, 本文改进后的框架在保证精度的同时极大提高了计算速率并降低了算力和存储需求。

6.2 展望

本文基于开源框架 DarkNet 进行扩展和优化。引入深度可分离卷积算法以及轻量神经网络模型 MobileNetV2，并使用了 ARM NEON 汇编指令及 OpenBLAS 对 GEMM 算法进行了优化，同时面向 ZYNQ 平台移植了 Linux 操作系统同时实现了图像输入采集单元，通过相关实验验证了整个系统在神经网络的计算当中，有效降低了神经网络计算参数量以及资源消耗，并且大幅提高了计算效率，在嵌入式神经网络技术应用方面取得一定进展。但是由于时间和条件的局限性，本文设计的神经网络计算框架还可以进一步完善，具体可以从以下几个方面进行研究：

- 1) 本文主要针对 ImageNet 数据集对部分网络进行了训练和测试，下一步的研究可在更多的数据集及网络模型上进行试验，验证本研究提出的方法是否具有普适性；
- 2) 除了紧致网络设计以及汇编指令和函数库可以优化神经网络的计算以外，还有一些优化方法本研究未来得及进行试验，下一步研究工作可试验其它优化方法，并进行对比，找到较好的优化方案；
- 3) 对于 ZYNQ SoC 嵌入式平台，本研究侧重于利用其 PS 端具有 ARM Cortex-a9 双核的优势进行计算加速，未对 PL 端进行可并行加速的试验，如何神经网络计算中的密集型运算移植到 PL 端加速将是下一步研究的重点。

参考文献

- 毕鹏程, 罗健欣, 陈卫卫, 等. 面向移动端的轻量化卷积神经网络结构[J]. 信息技术与网络安全, 2019(9).
- 戴雷燕, 冯杰, 董慧, 等. 面向嵌入式设备的深度学习物体检测优化算法[J]. 计算机系统应用, 2019,28(04):167-173.
- 何鹏程. 改进的卷积神经网络模型及其应用研究[D]. 大连理工大学, 2015.
- 纪荣嵘, 林绍辉, 晁飞, 等. 深度神经网络压缩与加速综述[J]. 计算机研究与发展, 2018,55(09):1871-1888.
- 李全. 面向ARM嵌入式平台的卷积神经网络前向加速研究[D]. 华中科技大学, 2019.
- 李世明. 基于云端辅助的嵌入式终端卷积神经网络模型更新框架技术研究[D]. 重庆大学, 2018.
- 李双峰. TensorFlow Lite:端侧机器学习框架[J]. 计算机研究与发展, 2020(9):1839-1853.
- 刘芾, 龙曼仪, 李茂军, 等. 基于轻量型卷积神经网络的交通标志识别[J]. 计算技术与自动化, 2020,v.39;No.156(04):117-123.
- 刘明. 基于深度学习的图像检索及图像语义特征研究[D]. 桂林电子科技大学, 2020.
- 聂阳, 王博文, 王宇鹏, 等. Zynq SOC嵌入式图像边缘检测系统设计[J]. 科技创新与应用, 2020.
- 任园园. 基于卷积神经网络的人脸活体检测算法研究[D]. 华南理工大学, 2020.
- 王立有. 基于深度学习的接触网腕臂支撑装置细小零部件缺失检测方案研究与实现[D]. 西南交通大学, 2019.
- 王韬. 嵌入式实时全景图像处理系统的设计[D]. 东南大学, 2017.
- 谢启荣. 基于DSP的嵌入式神经网络计算框架设计与实现[D]. 兰州大学, 2019.
- 张朝元, 邵高平, 汪洋. 基于Zynq-7000的嵌入式Linux移植[J]. 电子科技, 2018,31(01):9-11.
- 赵群, 刘托, 何亮亮, 等. 一种移动端异构开源神经网络计算框架MACE[J]. 信息技术与标准化, 2020,No.424(04):28-31.
- 郑冬. 基于轻量化卷积神经网络的目标检测[J]. 中国矿业大学, 2020.

- Abadi M, Barham P, Chen J, et al. TensorFlow: A system for large-scale machine learning[J]. USENIX Association, 2016.
- Aydonat U, O'Connell S, Capalija D, et al. An OpenCL(TM) Deep Learning Accelerator on Arria 10[J]. ACM, 2017.
- Bahrampour S, Ramakrishnan N, Schott L, et al. Comparative Study of Deep Learning Software Frameworks[J]. Computer ence, 2016.
- Bottleson J, Kim S Y, Andrews J, et al. clCaffe: OpenCL Accelerated Caffe for Convolutional Neural Networks: 2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2016[C].
- Chen J, Zhu Z, Li C, et al. Self-Adaptive Network Pruning[J]. 2019.
- Chollet F. Xception: Deep Learning with Depthwise Separable Convolutions: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017[C].
- Courbariaux M, Bengio Y. BinaryNet: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1[J]. 2016.
- Dicecco R, Lacey G, Vasiljevic J, et al. Caffeinated FPGAs: FPGA Framework For Convolutional Neural Networks: 2016 International Conference on Field-Programmable Technology (FPT), 2017[C].
- Han S M H D W. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding: ICLR, 2016[C].
- He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition[J]. IEEE, 2016.
- Highlander T, Rodriguez A. Very Efficient Training of Convolutional Neural Networks using Fast Fourier Transform and Overlap-and-Add[J]. 2016.
- Howard A G, Zhu M, Chen B, et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications[J]. 2017.
- Iandola F N, Han S, Moskewicz M W, et al. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size[J]. 2016.
- Jia D, Wei D, Socher R, et al. ImageNet: A large-scale hierarchical image database[J]. Proc of IEEE Computer Vision & Pattern Recognition, 2009:248-255.

- Jia Y. Learning Semantic Image Representations at a Large Scale[J]. 2014b.
- Jia Y, Shelhamer E, Donahue J, et al. Caffe: Convolutional Architecture for Fast Feature Embedding[J]. ACM, 2014a.
- Ko J H, Mudassar B, Na T, et al. Design of an Energy-Efficient Accelerator for Training of Convolutional Neural Networks using Frequency-Domain Computation[J]. IEEE, 2017:1-6.
- Kopuklu O, Kose N, Gunduz A, et al. Resource Efficient 3D Convolutional Neural Networks: 2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW), 2019[C].
- Krizhevsky A, Sutskever I, Hinton G. ImageNet Classification with Deep Convolutional Neural Networks: NIPS, 2012[C].
- Lecun Y, Bottou L. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998,86(11):2278-2324.
- Lu L, Yun L, Xiao Q, et al. Evaluating Fast Algorithms for Convolutional Neural Networks on FPGAs: IEEE International Symposium on Field-programmable Custom Computing Machines, 2017[C].
- Ma N, Zhang X, Zheng H T, et al. ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design[J]. Springer, Cham, 2018.
- Madaan D, Shin J, Hwang S J. Adversarial Neural Pruning with Latent Vulnerability Suppression[J]. 2019.
- Meng W, Gu Z, Zhang M, et al. Two-Bit Networks for Deep Learning on Resource-Constrained Embedded Devices[J]. 2017.
- Merolla P, Appuswamy R, Arthur J, et al. Deep neural networks are robust to weight binarization and other non-linear distortions[J]. 2016.
- Min Lin Q C S Y. Network In Network[N].
- Paszke A, Gross S, Chintala S, et al. Automatic differentiation in PyTorch[J]. 2017.
- Qian L, Xiao Q, Yun L. Enabling high performance deep learning networks on embedded systems: IECON 2017 - 43rd Annual Conference of the IEEE Industrial Electronics Society, 2017[C].
- Rastegari M, Ordonez V, Redmon J, et al. XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks[J]. Springer, Cham, 2016.

- Redmon J. Darknet: Open Source Neural Networks in C[EB/OL]. (2011-03-20)<https://pjreddie.com/darknet/>.
- Redmon J, Farhadi A. YOLOv3: An Incremental Improvement[J]. arXiv e-prints, 2018.
- Russakovsky O, Deng J, Su H, et al. ImageNet Large Scale Visual Recognition Challenge[J]. International Journal of Computer Vision, 2015,115(3):211-252.
- Sandler M, Howard A, Zhu M, et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks[J]. IEEE, 2018.
- Simonyan K, Zisserman A. Very Deep Convolutional Networks for Large-Scale Image Recognition[J]. Computer Science, 2014.
- Suda N, Chandra V, Dasika G, et al. Throughput-Optimized OpenCL-based FPGA Accelerator for Large-Scale Convolutional Neural Networks: Acn/sigda International Symposium, 2016[C].
- Wang Y, Zhang X, Xie L, et al. Pruning from Scratch[J]. 2019.
- Zhang J, Jing L. Improving the Performance of OpenCL-based FPGA Accelerator for Convolutional Neural Network: the 2017 ACM/SIGDA International Symposium, 2017[C].
- Zhang X, Zhou X, Lin M, et al. ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices[J]. 2017.
- Li X Y, Yin Z Y, Xu F L, et al. Design and implementation of neural network computing framework on Zynq SoC embedded platform [J]. The 10th International Conference of Information and Communication Technology (ICICT), 2020.

致 谢

转眼三年的研究生生涯即将结束，回想刚接到硕士录取通知书的那一刻的欣喜还历历在目。能够在中国科学院大学和沈阳计算所进行学习，实属荣幸之至。这三年是人生当中非常重要和美好的三年，回首既往，学习和思想上的提升、科研和学生工作的充实以及课余生活的多姿多彩，为我的人生赋予了浓墨重彩的一笔。这离不开各位老师同学，家人朋友的关心和帮助。

首先要感谢我的指导老师尹震宇研究员。尹老师渊博的学识、严谨对待学术的态度、爱岗敬业的精神、认真负责的处事风格，都是我学习的榜样！老师平日工作繁忙，但不管再忙都不忘监督我们的学习，经常利用休息时间对我们进行指导，我还记得好几次深夜被老师询问学习情况。每次有问题尹老师都会认真地对我们的研究进行细致地指导，耐心地审阅论文，提出宝贵意见和建议。在老师的指导和监督下，我的科研工作和论文才得以顺利完成。在这里向老师致以崇高的敬意和感谢！

我要感谢组织的培养以及研究生部各位老师的关怀和帮助。感谢组织的信任，感谢丁健老师、张玲老师、宁绍鹏老师平时的关心和支持，感谢刘卫强书记在国科大期间的培养，同时也感谢研究生部的其他几位老师，让我在研究生阶段的学生工作中得到了很大的历练，收获颇丰。

我要感谢实验室的小组成员，感谢徐福龙师兄、张飞青师姐为我的研究工作提供的思路和帮助，每次修改论文和答辩前师兄和师姐都不惜牺牲自己的时间对我进行耐心地指导，提出宝贵的意见。感谢王江波师弟、黄世青同学在技术方面提供的指导。感谢李岳师兄、谷艾师姐、徐光远同学提供的其他帮助。感谢可爱的室友们，在一起的日子互相关心，互相帮助，互相学习，互相鼓励，互相分享，让我感受到了家一样的温暖。感谢我的同学们，一起为了班级荣誉而拼搏，为了完成目标并肩作战，一起奋斗的三年充实而快乐。

我要感谢我的家人和朋友，每当我遇到困难的时候，你们总是无条件地支持和鼓励我。尤其要对我的母亲说一句：“您辛苦了！”在我成长的路上付出了

太多心血，总是默默地付出，您一直是我前进的动力！

最后，特别感谢各位专家、学者在百忙之中对本文的审阅，并恳请提出宝贵的意见和批评。

时光匆匆，青春如歌！硕士研究生阶段的学习时光即将结束，同时也意味着新的开始。回首过往，所有的汗水与泪水、荣誉与失败已经显得那么的微不足道。踏出校门，我依然会牢记“国有疑难可问谁？强国一代有我在！”

2021 年 6 月

